

VIETNAM NATIONAL UNIVERSITY
HO CHI MINH CITY UNIVERSITY OF TECHNOLOGY



Digital Systems

Lab 2

Instructor: Đoàn Ngọc Cẩm

Student	Student ID
Hồ Chí Dũng	2351016
Nguyễn Viết Thi	2451165
Trần Duy Anh	2351007

Ho Chi Minh City – March 2026

Laboratory 2:

FINITE STATE MACHINES

OBJECTIVES

- Design and implement digital circuits using FSM.
- Describe finite state machine (FSM) using two coding styles: logic-expression style and behavioral style.
- Analyze and implement the sequential system using multiple approaches, such as FSM method or by integrating flip-flops, registers, and combinational logic circuits.
- Download the circuit into the FPGA chip and test its functionality.

PREPARATION FOR LAB 2

- Complete Pre Lab 2 at home.
- Students are required to simulate all exercises in Pre Lab 2 at home. All results (including source code, waveforms, RTL viewer screenshots, etc.) must be captured and submitted to the instructor prior to the lab session.

If not, students will not participate in the lab and be considered absent this session.

REFERENCE

1. Intel FPGA training



Laboratory 2:

FINITE STATE MACHINES

EXPERIMENT 1

Objective: Implement an FSM circuit using System Verilog logic-expression style and download the circuit into the FPGA chip.

Requirement: Implement an FSM that recognizes two specific sequences of input symbols: four consecutive 1s or four consecutive 0s. The system has one input and one output. Whenever the input remains at 1 or at 0 for **four consecutive clock cycles**, the output must be set to 1; otherwise, =0.

Overlapping sequences are allowed. Therefore, if $w=1$ for five consecutive clock cycles, the output z will be equal to 1 after both the fourth and fifth cycles.

Figure 1 illustrates the required relationship between w and z , and Figure 2 presents the state diagram of the FSM.

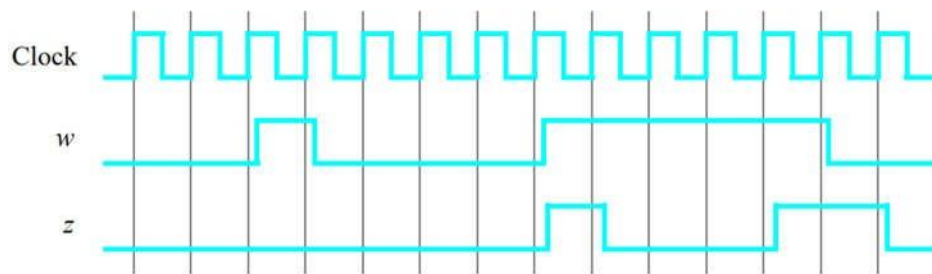


Figure 1: Required timing for the output z .

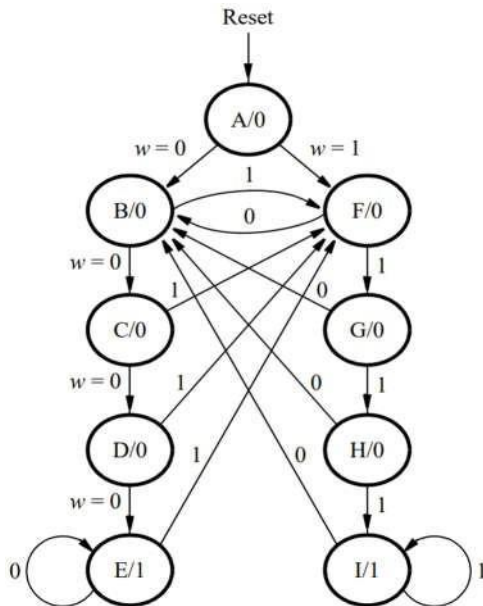
Students are required to implement the FSM circuit using the logic-expression style in System Verilog. The implementation must follow two state-encoding methods: one-hot encoding (Table 1a) and binary encoding (Table 1b).

The design must include synchronous reset signals, denoted as rst .



Laboratory 2:

FINITE STATE MACHINES



Name	State Code
	$y_8y_7y_6y_5y_4y_3y_2y_1y_0$
A	000000001
B	000000010
C	000000100
D	000001000
E	000010000
F	000100000
G	001000000
H	010000000
I	100000000

Name	State Code
	$y_3y_2y_1y_0$
A	0000
B	0001
C	0010
D	0011
E	0100
F	0101
G	0110
H	0111
I	1000

Figure 2: A state diagram for the FSM

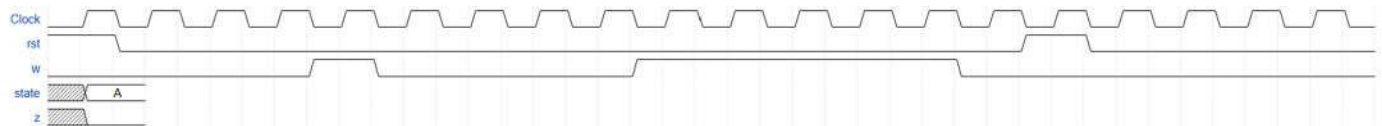
Table 1a: One-hot codes

Table 1b: Binary codes

Instruction:

1. Follow FSM circuit designed in Exercises 1 and 2 of Pre Lab 2, write the SystemVerilog assignment statements to implement the circuit using flip-flops and logic gates.

2. Based on your design, complete the timing diagram shown below.



3. Write a testbench to verify your design. The testbench must cover all cases shown in the timing diagram above.



Laboratory 2:

FINITE STATE MACHINES

4. Use the toggle switch SW0 as an active-low synchronous reset input for the FSM, use SW1 as the w input, and the push button KEY0 as the clock input which is applied manually. Use the red light LEDR9 as the output z and assign the state flip-flop outputs to the red lights LEDR8 to LEDR0.
5. Compile the project and then download the resulting circuit into the FPGA chip. Test the functionality of your design by applying the input sequences and observing the output LEDs. Make sure that the FSM properly transitions between states as displayed on the red LEDs, and that it produces the correct output values on LEDR9.
6. It is often desirable to set all flip-flop outputs to the value 0 in the reset state. Table 2 presents a modified one-hot state assignment in which the reset state (A) is encoded using all zeros. Since the reset state in binary encoding is also represented by all zeros, this choice of coding does not impact on the design. Create a modified version of your System Verilog code that implements this state assignment.
Compile your new circuit and test it.



Laboratory 2:

FINITE STATE MACHINES

Name	State Code
	$y_8y_7y_6y_5y_4y_3y_2y_1y_0$
A	000000000
B	000000011
C	000000101
D	000001001
E	000010001
F	000100001
G	001000001
H	010000001
I	100000001

Table 2: Modified one-hot codes for the FSM

Check: Your report must show two results:

- The next-state equations and the output equations of the FSM of your design.
- The waveform to prove the circuit works correctly.
- The result of RTL viewer.
- Experimental results obtained from running the design on the kit (the submitted photos must clearly show the student ID cards of all group members).



Laboratory 2:

FINITE STATE MACHINES

One-hot original:

The next state equation:

$$Y_0^+ = 0$$

$$Y_1^+ = (y_0 + y_5 + y_6 + y_7 + y_8) \cdot \overline{w}$$

$$Y_2^+ = y_1 \cdot \overline{w}$$

$$Y_3^+ = y_2 \cdot \overline{w}$$

$$Y_4^+ = (y_3 + y_4) \cdot \overline{w}$$

$$Y_5^+ = (y_0 + y_1 + y_2 + y_3 + y_4) \cdot w$$

$$Y_6^+ = y_5 \cdot w$$

$$Y_7^+ = y_6 \cdot w$$

$$Y_8^+ = (y_7 + y_8) \cdot w$$

The output equation:

$$z = y_4 + y_8$$

One-hot modified:

The next state equation:

$$Y_0 = 1$$

$$Y_1 = (\overline{y_0} + y_5 + y_6 + y_7 + y_8) \cdot \overline{w}$$

$$Y_2 = y_1 \cdot \overline{w}$$

$$Y_3 = y_2 \cdot \overline{w}$$

$$Y_4 = (y_3 + y_4) \cdot \overline{w}$$

$$Y_5 = (\overline{y_0} + y_1 + y_2 + y_3 + y_4) \cdot w$$

$$Y_6 = y_5 \cdot w$$

$$Y_7 = y_6 \cdot w$$

$$Y_8 = (y_7 + y_8) \cdot w$$

The output equation:

$$z = y_4 + y_8$$



Laboratory 2:

FINITE STATE MACHINES

Binary:

The next-state equations:

$$Y_0 = (\overline{y_3} \cdot \overline{y_2} \cdot w) + (\overline{y_3} \cdot \overline{y_0} \cdot w) + (\overline{y_2} \cdot \overline{y_1} \cdot \overline{y_0} \cdot \overline{w}) + (\overline{y_3} \cdot y_2 \cdot y_0 \cdot \overline{w}) + (\overline{y_3} \cdot y_1 \cdot \overline{y_0})$$

$$Y_1 = (\overline{y_3} \cdot \overline{y_2} \cdot \overline{y_1} \cdot y_0 \cdot \overline{w}) + (\overline{y_3} \cdot \overline{y_2} \cdot y_1 \cdot \overline{y_0} \cdot \overline{w}) + (\overline{y_3} \cdot y_2 \cdot \overline{y_1} \cdot y_0 \cdot w) + (\overline{y_3} \cdot y_2 \cdot y_1 \cdot \overline{y_0} \cdot w)$$

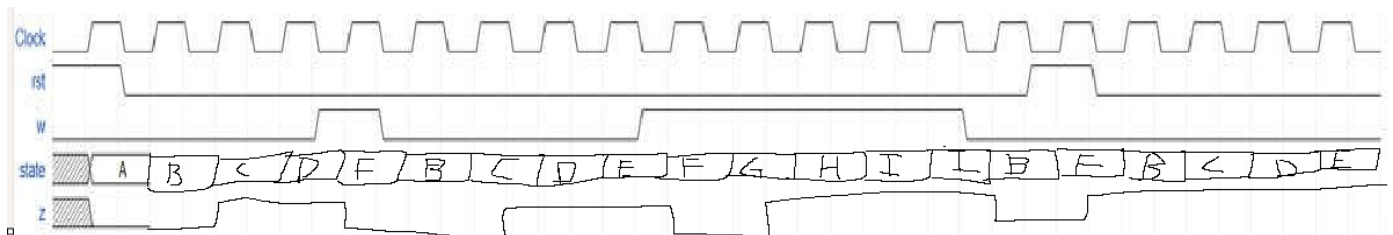
$$Y_2 = (\overline{y_3} \cdot \overline{y_1} \cdot w) + (\overline{y_3} \cdot \overline{y_0} \cdot w) + (\overline{y_3} \cdot y_2 \cdot \overline{y_1} \cdot \overline{y_0}) + (\overline{y_3} \cdot \overline{y_2} \cdot y_1 \cdot y_0)$$

$$Y_3 = (\overline{y_3} \cdot y_2 \cdot y_1 \cdot y_0 \cdot w) + (y_3 \cdot \overline{y_2} \cdot \overline{y_1} \cdot \overline{y_0} \cdot w)$$

The output equations:

$$z = (\overline{y_3} \cdot y_2 \cdot \overline{y_1} \cdot \overline{y_0}) + (y_3 \cdot \overline{y_2} \cdot \overline{y_1} \cdot \overline{y_0})$$

Timing diagram



Laboratory 2:

FINITE STATE MACHINES

1a. One-hot codes

Code

```
module register #(parameter n=9)
(
    input rst, clk,
    input [(n-1):0] D,
    output logic [(n-1):0] Q
);
    always_ff @(posedge clk)
        begin
            if (!rst)
                Q <= 9'b000000001;
            else
                Q <= D;
            end
        endmodule

module comb_logic (
    input [8:0] p_state,
    input in,
    output [8:0] n_state,
    output out
);
    assign n_state[0] = 1'b0;
    assign n_state[1] = (p_state[0] | p_state[5] | p_state[6] | p_state[7] | p_state[8]) & (~in);
    assign n_state[2] = p_state[1] & (~in);
    assign n_state[3] = p_state[2] & (~in);
    assign n_state[4] = (p_state[3] | p_state[4]) & (~in);
    assign n_state[5] = (p_state[0] | p_state[1] | p_state[2] | p_state[3] | p_state[4]) & in;
    assign n_state[6] = p_state[5] & in;
    assign n_state[7] = p_state[6] & in;
    assign n_state[8] = (p_state[7] | p_state[8]) & in;

    assign out = (p_state[4]) | (p_state[8]);
endmodule

module lab2_tn1_hot
(
    input [1:0] SW,
    input [0:0] KEY,
    output [9:0] LEDR
);
```



Laboratory 2:

FINITE STATE MACHINES

```
);  
logic [8:0] a, b;  
comb_logic comb_inst (  
    .in (SW[1]),  
    .p_state (a),  
    .n_state (b),  
    .out(LED[9])  
);  
register reg_inst(  
    .Q(a),  
    .D(b),  
    .clk(KEY[0]),  
    .rst(SW[0])  
);  
assign LED[8:0] = a;  
endmodule
```

Testbench

```
`timescale 1ns / 1ps  
  
module lab2_tn1_hot_tb();  
  
    // Khai báo các tín hiệu kết nối với module cần test (DUT)  
    logic clk;  
    logic rst; // Reset tích cực mức thấp (dựa theo hình ảnh)  
    logic w;  
    logic z;  
    logic [8:0] state; // Dùng để quan sát trạng thái (Y)  
  
    // Gọi module (DUT - Device Under Test)  
    // Lưu ý: Thay "prelab2_tn1" bằng tên module thực tế của bạn nếu khác  
    lab2_tn1_hot DUT (  
        .KEY(clk),           // Nối tín hiệu clk của testbench vào KEY[0]  
        .SW({w, rst}),       // Nối w vào SW[1] và rst vào SW[0]  
        .LED({z, state})     // Nối ngõ ra z vào LED[9] và trạng thái vào LED[8:0]  
    );  
  
    // Tạo xung Clock (Chu kỳ 10ns -> Tần số 100MHz)  
    initial begin  
        clk = 0;  
        forever #5 clk = ~clk;  
    end  
end
```



Laboratory 2:

FINITE STATE MACHINES

```
// Khởi tạo tín hiệu kích thích (Stimulus)
initial begin
    // 1. Khởi tạo ban đầu
    rst = 1'b0;
    w = 1'b0;

    // 2. Kích hoạt Reset (Active-low) ở đầu mô phỏng
    @(negedge clk); // Đợi cạnh xuống của clock
    rst = 1'b1;      // Kéo rst xuống 0 để reset về trạng thái A

    @(negedge clk);
    rst = 1'b1;      // Nhả reset (lên 1), FSM bắt đầu hoạt động

    // 3. w = 0 trong khoảng 2 chu kỳ tiếp theo (tổng là 3 chu kỳ đầu)
    @(negedge clk);
    @(negedge clk);

    // 4. w = 1 trong 1 chu kỳ
    w = 1'b1;
    @(negedge clk);

    // 5. w = 0 trong 3 chu kỳ
    w = 1'b0;
    @(negedge clk);
    @(negedge clk);
    @(negedge clk);
    @(negedge clk);

    // 6. w = 1 trong 5 chu kỳ liên tiếp
    // Đủ 4 chu kỳ thì z sẽ nhảy lên 1, chu kỳ thứ 5 z vẫn giữ mức 1
    w = 1'b1;
    @(negedge clk); // 1
    @(negedge clk); // 2
    @(negedge clk); // 3
    @(negedge clk); // 4 -> Sau cạnh lên tiếp theo z sẽ = 1
    @(negedge clk); // 5 -> z tiếp tục giữ 1

    // 7. w = 0 trở lại, ngắt chuỗi 1 -> z sẽ rớt xuống 0
    w = 1'b0;
    @(negedge clk);
    rst = 1'b0;      // Kéo rst xuống 0
    @(negedge clk);
    rst = 1'b1;      // Nhả reset
    @(negedge clk);
    @(negedge clk);

    // 8. Test thử reset giữa chừng
```

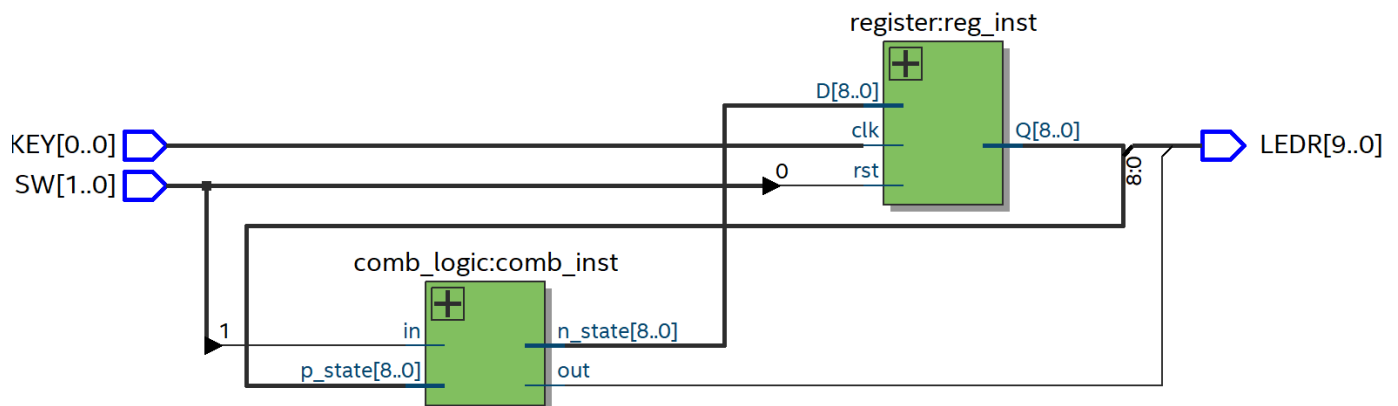


Laboratory 2:

FINITE STATE MACHINES

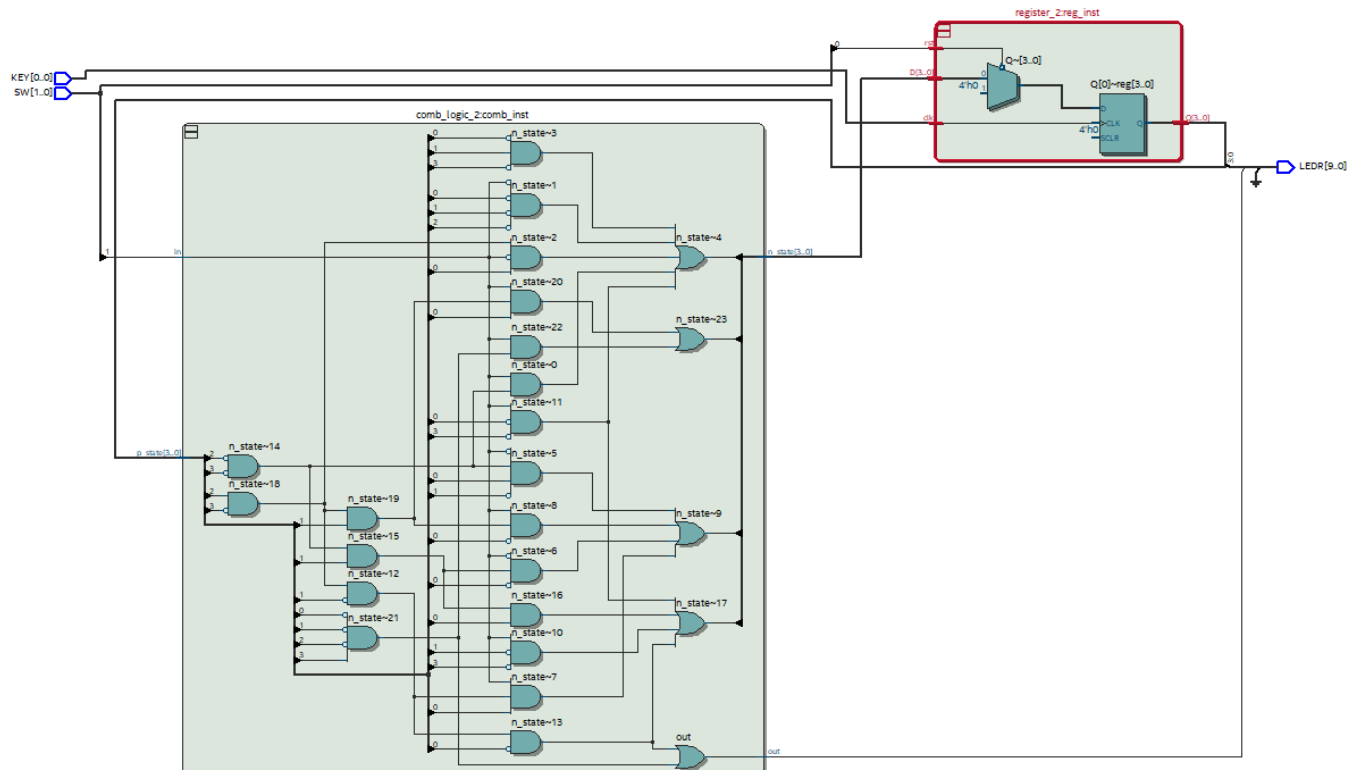
```
// Đợi thêm vài chu kỳ rồi kết thúc mô phỏng
@ (negedge clk) ;
@ (negedge clk) ;
$stop;           // Dừng mô phỏng
end
endmodule
```

RTL

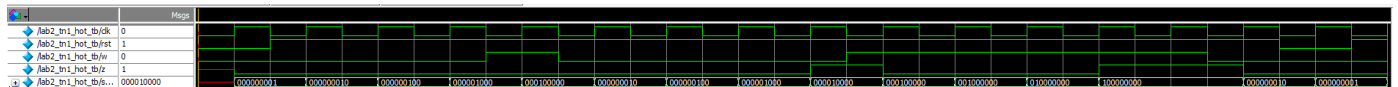


Laboratory 2:

FINITE STATE MACHINES



Simulation



Experimental results VIDEO:

https://drive.google.com/file/d/1VnLeFIg41EmirQxTdc-rTDWmSHrPCHkH/view?usp=drive_link



Laboratory 2:

FINITE STATE MACHINES

1b. Binary codes

```
module register_2
(
    input rst, clk,
    input [3:0] D,
    output logic [3:0] Q
);
always_ff @(posedge clk)
begin
    if (!rst)
        Q <= 3'b0000;
    else
        Q <= D;
    end
endmodule

module comb_logic_2 (
    input [3:0] p_state,
    input in,
    output [3:0] n_state,
    output out
);
/*
assign n_state[0] = (~p_state[3] & ~p_state[2] & ~p_state[1] &
    ~p_state[0] & ~in)
*/

assign n_state[0] = (~p_state[3] & ~p_state[2] & in)
    | (~p_state[3] & ~p_state[0] & in)
    | (~p_state[2] & ~p_state[1] & ~p_state[0] & ~in)
    | (~p_state[3] & p_state[2] & p_state[0] & ~in)
    | (~p_state[3] & p_state[1] & ~p_state[0]);

assign n_state[1] = (~p_state[3] & ~p_state[2] & ~p_state[1] &
    p_state[0] & ~in)
    | (~p_state[3] & ~p_state[2] & p_state[1] &
    ~p_state[0] & ~in)
    | (~p_state[3] & p_state[2] & ~p_state[1] &
    p_state[0] & in)
    | (~p_state[3] & p_state[2] & p_state[1] &
    ~p_state[0] & in);
```



Laboratory 2:

FINITE STATE MACHINES

```

assign n_state[2] = (~p_state[3] & ~p_state[1] & in)
| (~p_state[3] & ~p_state[0] & in)
| (~p_state[3] & p_state[2] & ~p_state[1] & ~p_state[0])
| (~p_state[3] & ~p_state[2] & p_state[1] & p_state[0]);
assign n_state[3] = (~p_state[3] & p_state[2] & p_state[1] &
p_state[0] & in)
| (p_state[3] & ~p_state[2] & ~p_state[1] & ~p_state[0] &
in);
assign out
~p_state[0] ) = (~p_state[3] & p_state[2] & ~p_state[1] &
| (p_state[3] & ~p_state[2] & ~p_state[1] & ~p_state[0] );
endmodule

module lab2_tn1_binary
(
    input [0:0]KEY,
    input [1:0]SW,
    output [9:0] LEDR
);
logic [3:0] a, b;
comb_logic_2 comb_inst (
    .in (SW[1]),
    .p_state (a),
    .n_state (b),
    .out(LEDR[9])
);
register_2 reg_inst(
    .Q(a),
    .D(b),
    .clk(KEY[0]),
    .rst(SW[0])
);
assign LEDR[3:0]= a;
assign LEDR[8:4] = 5'b00000;
endmodule

```



Laboratory 2:

FINITE STATE MACHINES

Testbench

```
`timescale 1ns / 1ps

module lab2_tn1_binary_tb();

    // Khai báo các tín hiệu kết nối với module cần test (DUT)
    logic clk;
    logic rst; // Reset tích cực mức thấp (dựa theo hình ảnh)
    logic w;
    logic z;
    logic [3:0] state; // Dùng để quan sát trạng thái (Y)
    logic [9:0] ledr; // Khai báo mảng 10 bit, để có thể cho z = LEDR9, state = LEDR[3:0]

    assign z = ledr[9];
    assign state = ledr[3:0];
    lab2_tn1_binary DUT (
        .KEY(clk),          // Nối tín hiệu clk của testbench vào KEY[0]
        .SW({w, rst}),      // Nối w vào SW[1] và rst vào SW[0]
        .LEDR(ledr)         // Nối ngõ ra z vào LEDR[9] và trạng thái vào LEDR[3:0]
    );

    // Tạo xung Clock (Chu kỳ 10ns -> Tần số 100MHz)
    initial begin
        clk = 0;
        forever #5 clk = ~clk;
    end

    // Khởi tạo tín hiệu kích thích (Stimulus)
    initial begin
        // 1. Khởi tạo ban đầu
        rst = 1'b0;
        w = 1'b0;

        // 2. Kích hoạt Reset (Active-low) ở đầu mô phỏng
        @(negedge clk); // Đợi cạnh xuống của clock
        rst = 1'b1;     // Kéo rst xuống 0 để reset về trạng thái A

        @(negedge clk);
        rst = 1'b1;     // Nhả reset (lên 1), FSM bắt đầu hoạt động

        // 3. w = 0 trong khoảng 2 chu kỳ tiếp theo (tổng là 3 chu kỳ đầu)
        @(negedge clk);
        @(negedge clk);

        // 4. w = 1 trong 1 chu kỳ
```



Laboratory 2:

FINITE STATE MACHINES

```
w = 1'b1;
@(negedge clk);

// 5. w = 0 trong 3 chu kỳ
w = 1'b0;
@(negedge clk);
@(negedge clk);
@(negedge clk);
@(negedge clk);

// 6. w = 1 trong 5 chu kỳ liên tiếp
// Dù 4 chu kỳ thì z sẽ nhảy lên 1, chu kỳ thứ 5 z vẫn giữ mức 1
w = 1'b1;
@(negedge clk); // 1
@(negedge clk); // 2
@(negedge clk); // 3
@(negedge clk); // 4 -> Sau cạnh lên tiếp theo z sẽ = 1
@(negedge clk); // 5 -> z tiếp tục giữ 1

// 7. w = 0 trở lại, ngắt chuỗi 1 -> z sẽ rớt xuống 0
w = 1'b0;
@(negedge clk);
    rst = 1'b0;      // Kéo rst xuống 0
@(negedge clk);
rst = 1'b1;          // Nhà reset
@(negedge clk);
@(negedge clk);

// 8. Test thử reset giữa chừng

// Đợi thêm vài chu kỳ rồi kết thúc mô phỏng
@(negedge clk);
@(negedge clk);
$stop;              // Dừng mô phỏng
end

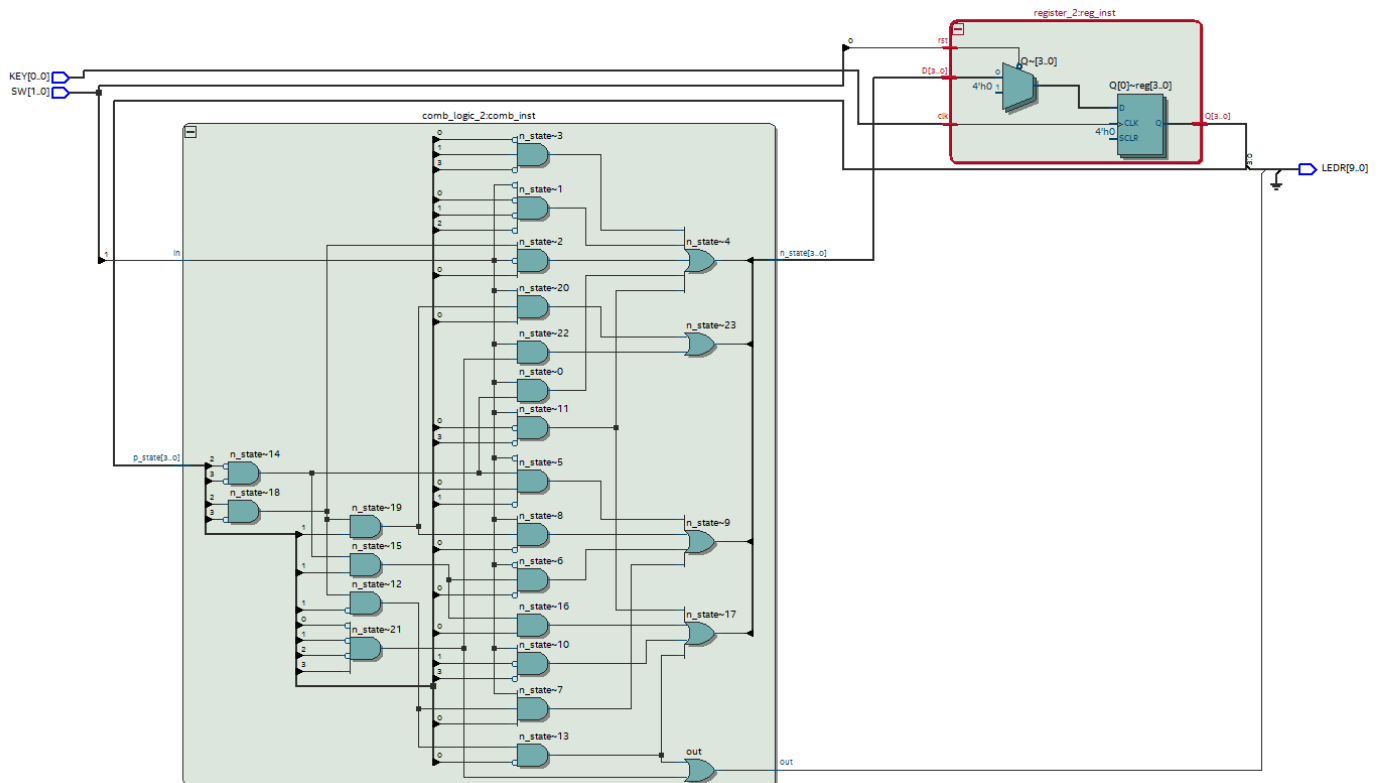
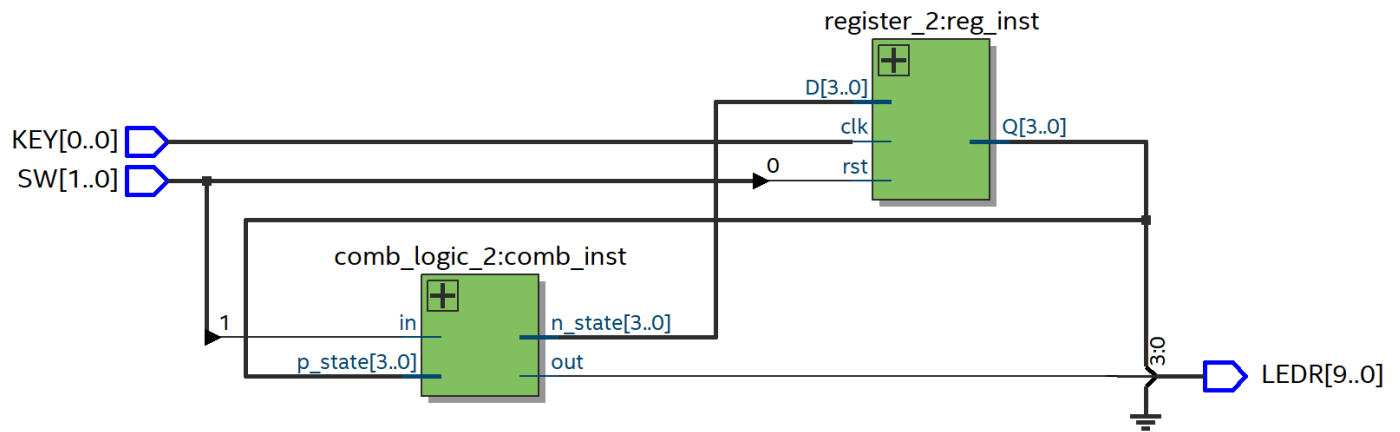
endmodule
```



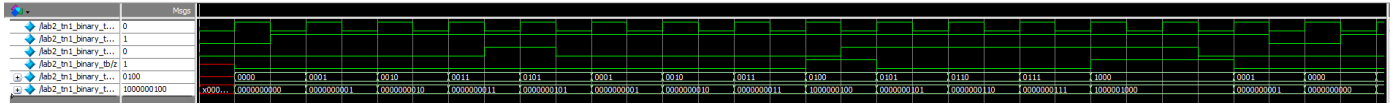
Laboratory 2:

FINITE STATE MACHINES

RTL



Simulation



https://drive.google.com/file/d/1VNYGEWQz1jV8SI3PZtQJ8_5SyUUUndC_L/view?usp=drive_link

Laboratory 2:

FINITE STATE MACHINES

1c. Modified One-hot codes

Code

```
module register6 #(parameter n=9)
(
    input rst, clk,
    input [(n-1):0] D,
    output logic [(n-1):0] Q
);
    always_ff @(posedge clk)
        begin
            if (!rst)
                Q <= 9'b000000000;
            else
                Q <= D;
            end
        endmodule

module comb_logic6 (
    input [8:0] p_state,
    input in,
    output [8:0] n_state,
    output out
);
    assign n_state[0] = 1'b1;
    assign n_state[1] = (~p_state[0] | p_state[5] | p_state[6] | p_state[7] | p_state[8])
    & (~in);
    assign n_state[2] = p_state[1] & (~in);
    assign n_state[3] = p_state[2] & (~in);
    assign n_state[4] = (p_state[3] | p_state[4]) & (~in);
    assign n_state[5] = (~p_state[0] | p_state[1] | p_state[2] | p_state[3] | p_state[4])
    & in;
    assign n_state[6] = p_state[5] & in;
    assign n_state[7] = p_state[6] & in;
    assign n_state[8] = (p_state[7] | p_state[8]) & in;

    assign out = (p_state[4]) | (p_state[8]);
endmodule

module lab2_tn1_6
(
    input [1:0] SW,
    input [0:0] KEY,
    output [9:0] LEDR

```



Laboratory 2:

FINITE STATE MACHINES

```
);  
logic [8:0] a, b;  
comb_logic6 comb_inst (  
    .in (SW[1]),  
    .p_state (a),  
    .n_state (b),  
    .out(LED[9])  
);  
register6 reg_inst(  
    .Q(a),  
    .D(b),  
    .clk(KEY[0]),  
    .rst(SW[0])  
);  
assign LED[8:0] = a;  
endmodule
```



Laboratory 2:

FINITE STATE MACHINES

Testbench

```
`timescale 1ns / 1ps

module lab2_tn1_6_tb();

    // Khai báo các tín hiệu kết nối với module cần test (DUT)
    logic clk;
    logic rst; // Reset tích cực mức thấp (dựa theo hình ảnh)
    logic w;
    logic z;
    logic [8:0] state; // Dùng để quan sát trạng thái (Y)

    // Gọi module (DUT - Device Under Test)
    // Lưu ý: Thay "prelab2_tn1" bằng tên module thực tế của bạn nếu khác
    lab2_tn1_6 DUT (
        .KEY(clk),           // Nối tín hiệu clk của testbench vào KEY[0]
        .SW({w, rst}),       // Nối w vào SW[1] và rst vào SW[0]
        .LEDR({z, state})    // Nối ngõ ra z vào LEDR[9] và trạng thái vào
LEDR[8:0]
    );

    // Tạo xung Clock (Chu kỳ 10ns -> Tần số 100MHz)
    initial begin
        clk = 0;
        forever #5 clk = ~clk;
    end

    // Khởi tạo tín hiệu kích thích (Stimulus)
    initial begin
        // 1. Khởi tạo ban đầu
        rst = 1'b0;
        w = 1'b0;

        // 2. Kích hoạt Reset (Active-low) ở đầu mô phỏng
        @(negedge clk); // Đợi cạnh xuống của clock
        rst = 1'b1;     // Kéo rst xuống 0 để reset về trạng thái A

        @(negedge clk);
        rst = 1'b1;     // Nhả reset (lên 1), FSM bắt đầu hoạt động

        // 3. w = 0 trong khoảng 2 chu kỳ tiếp theo (tổng là 3 chu kỳ đầu)
        @(negedge clk);
        @(negedge clk);

        // 4. w = 1 trong 1 chu kỳ
```



Laboratory 2:

FINITE STATE MACHINES

```
w = 1'b1;
@(negedge clk);

// 5. w = 0 trong 3 chu kỳ
w = 1'b0;
@(negedge clk);
@(negedge clk);
@(negedge clk);
@(negedge clk);

// 6. w = 1 trong 5 chu kỳ liên tiếp
// Dù 4 chu kỳ thì z sẽ nhảy lên 1, chu kỳ thứ 5 z vẫn giữ mức 1
w = 1'b1;
@(negedge clk); // 1
@(negedge clk); // 2
@(negedge clk); // 3
@(negedge clk); // 4 -> Sau cạnh lên tiếp theo z sẽ = 1
@(negedge clk); // 5 -> z tiếp tục giữ 1

// 7. w = 0 trở lại, ngắt chuỗi 1 -> z sẽ rớt xuống 0
w = 1'b0;
@(negedge clk);
    rst = 1'b0;      // Kéo rst xuống 0
@(negedge clk);
rst = 1'b1;          // Nhà reset
@(negedge clk);
@(negedge clk);

// 8. Test thử reset giữa chừng

// Đợi thêm vài chu kỳ rồi kết thúc mô phỏng
@(negedge clk);
@(negedge clk);
$stop;              // Dừng mô phỏng
end

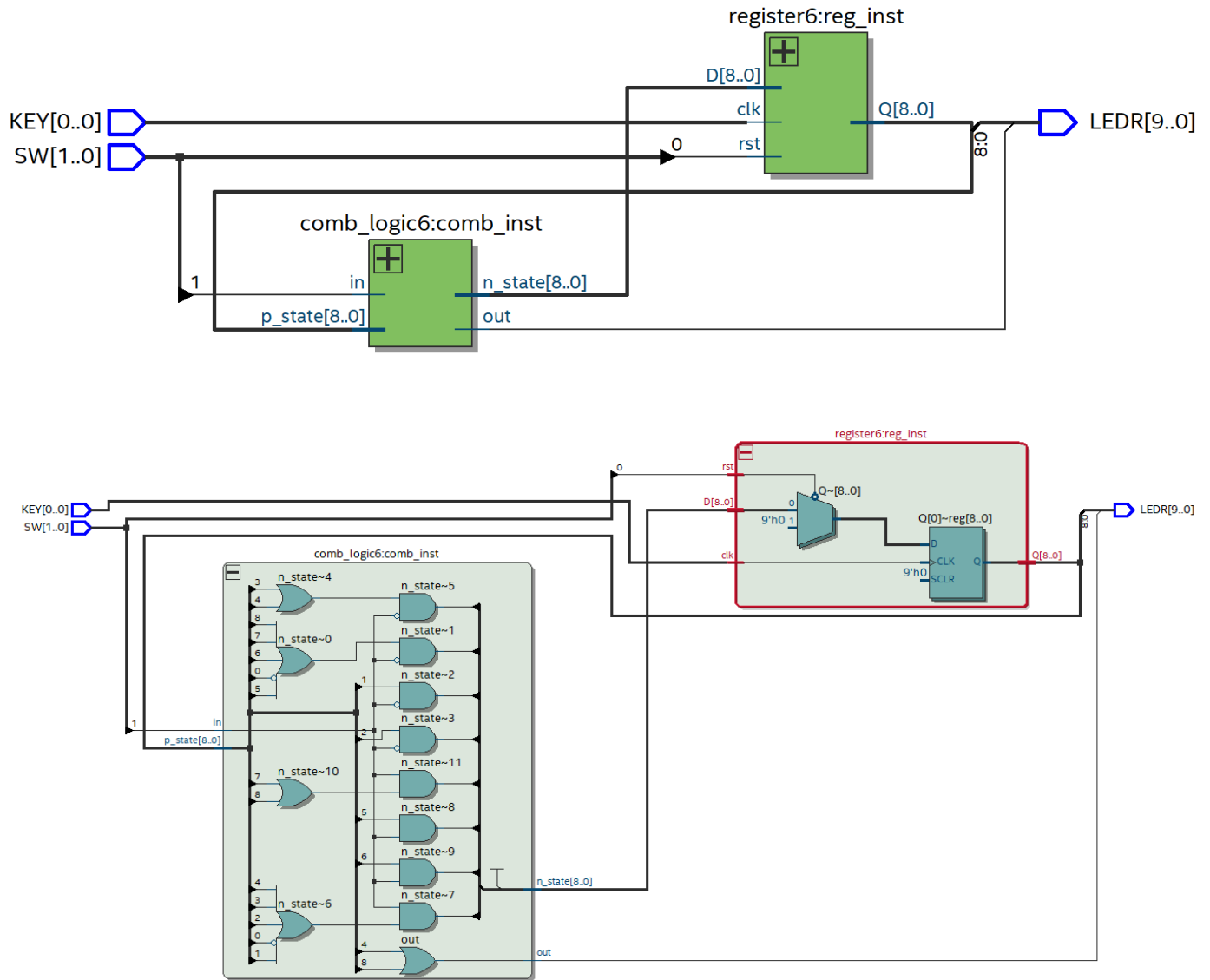
endmodule
```



Laboratory 2:

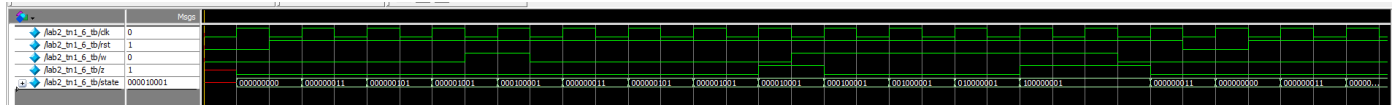
FINITE STATE MACHINES

RTL



Laboratory 2: FINITE STATE MACHINES

Simulation



Experimental results VIDEO:

[https://drive.google.com/file/d/1F67szomsErC55KGDeAIXrZjsUADrQ_4F/view?usp=drive link](https://drive.google.com/file/d/1F67szomsErC55KGDeAIXrZjsUADrQ_4F/view?usp=drive_link)

Laboratory 2:

FINITE STATE MACHINES

EXPERIMENT 2

Objective: Implement an FSM circuit using System Verilog behavioral style and download the circuit into the FPGA chip.

Requirement: Implement the FSM given in experiment 1, using behavioral style of System Verilog code. The implementation must follow two state-encoding methods: one-hot encoding (Table 1a) and binary encoding (Table 1b).

Instruction:

1. Based on the behavioral System Verilog coding style used in Exercises 1 and 2 of

Pre Lab 2, develop the System Verilog code and compile the project.

2. It is necessary to explicitly tell the Synthesis tool in Quartus that you wish to have the finite state machine implemented using the state assignment specified in your Verilog code. If you do not explicitly give this setting to Quartus, the Synthesis tool will automatically use a state assignment of its own choosing, and it will ignore the state codes specified in your System Verilog code. To make this setting, choose **Assignments > Settings** in Quartus, and click on the **Compiler Settings** item on the left side of the window, then click on the **Advanced Settings (Synthesis)** button. As indicated in Figure 3, change the parameter **State Machine Processing** to the setting **User-Encoded**.



Laboratory 2:

FINITE STATE MACHINES

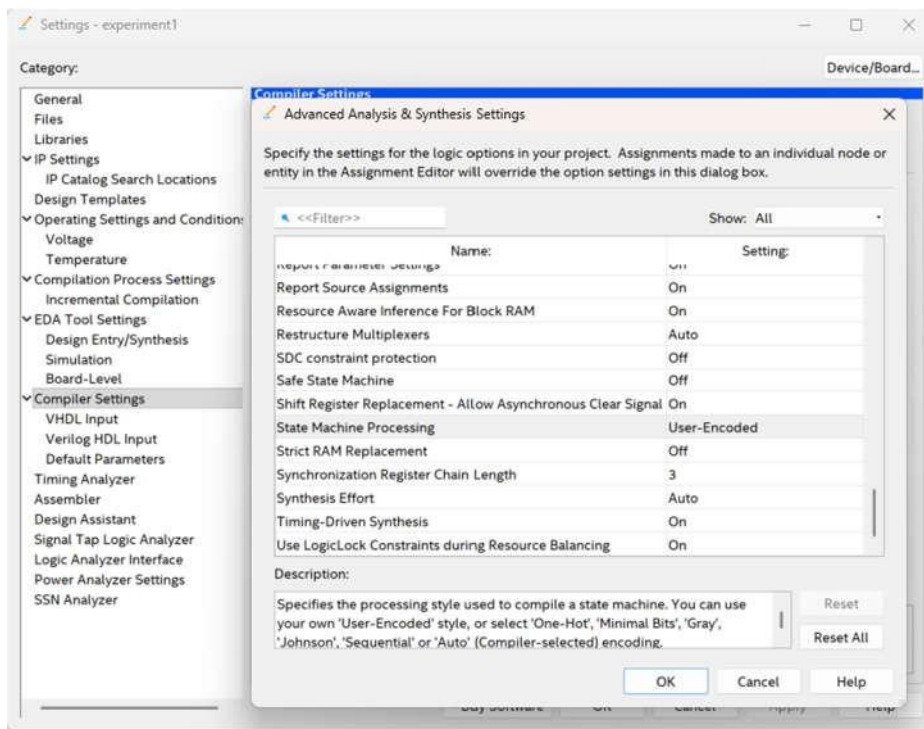


Figure 3: Specifying the state assignment method in Quartus.

3. Examine the circuit produced by Quartus open the **RTL Viewer** tool. Double-click on the box shown in the circuit that represents the finite state machine and determine whether the state diagram that it shows properly corresponds to the one in Figure 2.



Laboratory 2:

FINITE STATE MACHINES

4. To see the state codes used for your FSM: open the **Compilation Report**
→ **Analysis and Synthesis** section → **State Machines**.
5. Use the same switches, push buttons, and lights that were used in previous experiments.
6. Download the circuit into the FPGA chip and test its functionality.

Check: Your report must show two results:

- The waveform to prove the circuit works correctly.
- The result of RTL viewer.
- The result of State Machines in Compilation Report.
- Experimental results obtained from running the design on the kit (the submitted photos must clearly show the student ID cards of all group members).



Laboratory 2:

FINITE STATE MACHINES

One hot Code:

```
module lab2_tn2_hot (
    input [1:0] SW,
    input [0:0] KEY,
    output [9:0] LEDR
) ;
// 1. State declaration (one-hot encoding)
parameter
A = 9'b000000001,
B = 9'b000000010,
C = 9'b000000100,
D = 9'b000001000,
E = 9'b000010000,
F = 9'b000100000,
G = 9'b001000000,
H = 9'b010000000,
I = 9'b100000000;
logic [8:0] present_state , next_state ;
// 2. State register
always_ff @ ( posedge KEY[0] , posedge SW[0] ) begin /* FF description*/
if (SW[0])
present_state <= A;
else
present_state <= next_state;
end
// 3. Next-state combinational logic
always_comb /*next state transition*/
begin
next_state = present_state; // default state is the same
case ( present_state )
A : if ( SW[1] ) next_state = F;
else next_state = B;
B : if ( SW[1] ) next_state = F;
else next_state = C;
C : if ( SW[1] ) next_state = F;
else next_state = D;
D : if ( SW[1] ) next_state = F;
else next_state = E;
E : if ( SW[1] ) next_state = F;
else next_state = E;
F : if ( SW[1] ) next_state = G;
else next_state = B;
G : if ( SW[1] ) next_state = H;
else next_state = B;
end
```



Laboratory 2:

FINITE STATE MACHINES

```
H : if ( SW[1] ) next_state = I;
else next_state = B;
I : if ( SW[1] ) next_state = I;
else next_state = B;
default: next_state = A;
endcase
end
// 4. Output logic
assign LEDR[9] = (present_state == E) || (present_state == I);
assign LEDR[8:0] = present_state;
endmodule
```

Testbench:

```
`timescale 1ns / 1ps

module lab2_tn2_hot_tb();

    // Khai báo các tín hiệu kết nối
    logic clk;
    logic rst; // Đổi tên thành rst_n để nhớ đây là reset tích cực mức thấp (SW0)
    logic w;
    logic z;
    logic [8:0] state; // Dùng 9 bit để bao quát được cả thiết kế One-Hot

    // Gọi module (DUT - Device Under Test)
    lab2_tn2_hot dut (
        .KEY(clk),           // Nối tín hiệu clk vào KEY[0]
        .SW({w, rst}),       // Nối w vào SW[1] và rst_n vào SW[0]
        .LEDR({z, state})    // Nối ngõ ra z vào LEDR[9] và trạng thái vào LEDR[8:0]
    );

    // Tạo xung Clock (Chu kỳ 10ns -> Tần số 100MHz)
    initial begin
        clk = 0;
        forever #5 clk = ~clk;
    end

    // Khởi tạo tín hiệu kích thích (Stimulus)
    initial begin
        // 1. Khởi tạo ban đầu
```



Laboratory 2:

FINITE STATE MACHINES

```
rst = 1'b1;           // Kéo rst lên 1 để reset về trạng thái A
w = 1'b0;

// 2. Kích hoạt Reset (Active-low) ở đầu mô phỏng
@(negedge clk); // Đợi cạnh xuống của clock
rst = 1'b0;

@(negedge clk);
rst = 1'b0;           // FSM bắt đầu hoạt động

// 3. w = 0 trong khoảng 2 chu kỳ tiếp theo (tổng là 3 chu kỳ đầu)
@(negedge clk);
@(negedge clk);

// 4. w = 1 trong 1 chu kỳ
w = 1'b1;
@(negedge clk);

// 5. w = 0 trong 3 chu kỳ
w = 1'b0;
@(negedge clk);
@(negedge clk);
@(negedge clk);
@(negedge clk);

// 6. w = 1 trong 5 chu kỳ liên tiếp
// Dù 4 chu kỳ thì z sẽ nhảy lên 1, chu kỳ thứ 5 z vẫn giữ mức 1
w = 1'b1;
@(negedge clk); // 1
@(negedge clk); // 2
@(negedge clk); // 3
@(negedge clk); // 4 -> Sau cạnh lên tiếp theo z sẽ = 1
@(negedge clk); // 5 -> z tiếp tục giữ 1

// 7. w = 0 trở lại, ngắt chuỗi 1 -> z sẽ rớt xuống 0
w = 1'b0;
@(negedge clk);
    rst = 1'b1;           // Kéo rst lên 1
@(negedge clk);
rst = 1'b0;           // Máy chạy bth
@(negedge clk);
@(negedge clk);

// 8. Test thử reset giữa chừng
```

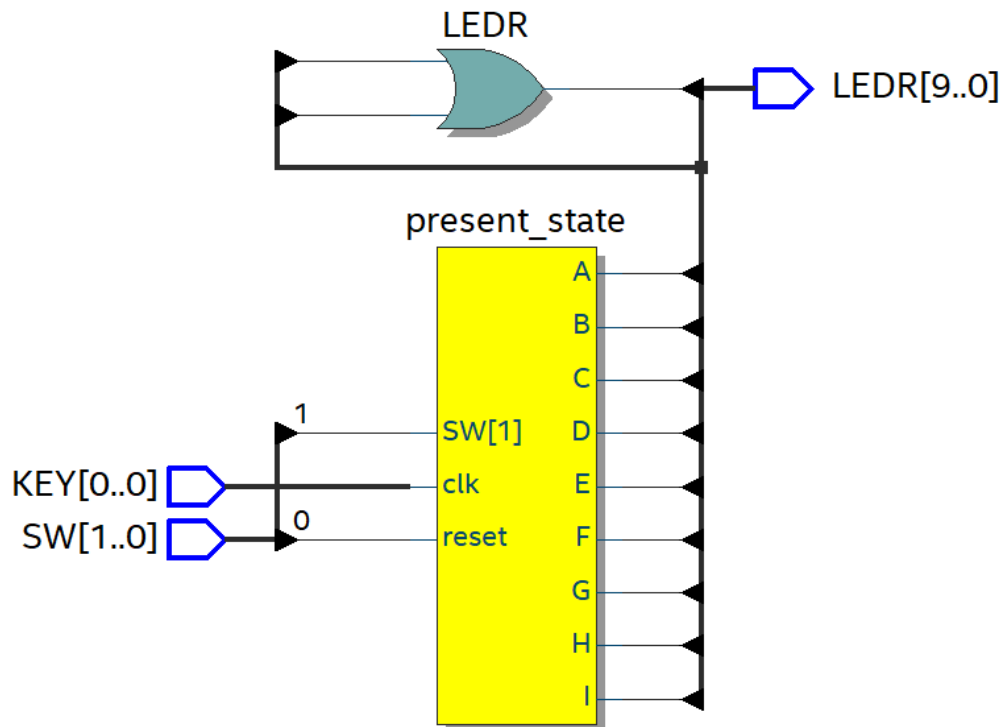


Laboratory 2:

FINITE STATE MACHINES

```
// Đợi thêm vài chu kỳ rồi kết thúc mô phỏng  
@(negedge clk);  
@(negedge clk);  
$stop;           // Dừng mô phỏng  
end  
endmodule
```

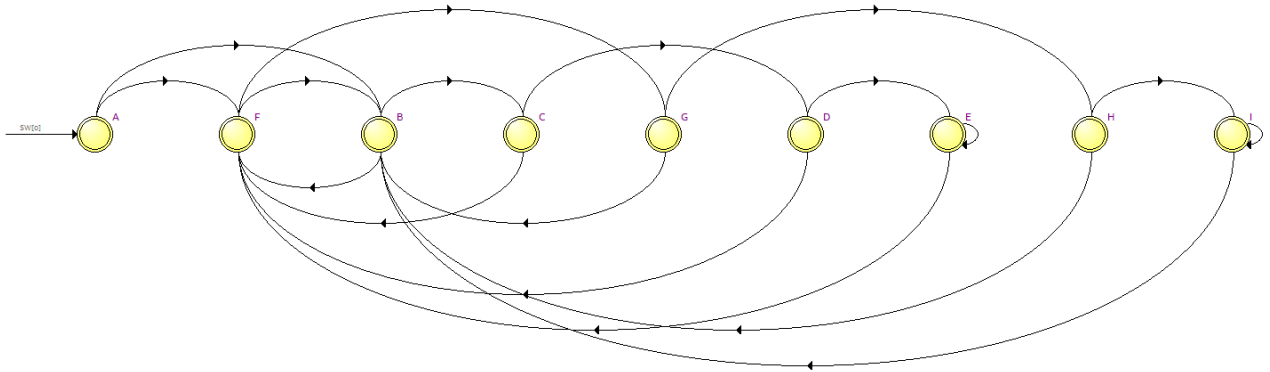
RTL



Laboratory 2:

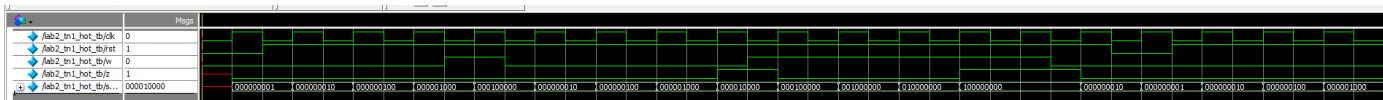
FINITE STATE MACHINES

State machine



	Name	present_state~12	present_state~11	present_state~10	present_state~9	present_state~8	present_state~7	present_state~6	present_state~5	p
1	present_state.A	0	0	0	0	0	0	0	0	0
2	present_state.B	0	0	0	0	0	0	0	1	1
3	present_state.C	0	0	0	0	0	0	1	0	1
4	present_state.D	0	0	0	0	0	1	0	0	1
5	present_state.E	0	0	0	0	1	0	0	0	1
6	present_state.F	0	0	0	1	0	0	0	0	1
7	present_state.G	0	0	1	0	0	0	0	0	1
8	present_state.H	0	1	0	0	0	0	0	0	1
9	present_state.I	1	0	0	0	0	0	0	0	1

Simulation



Experimental results VIDEO:

https://drive.google.com/file/d/1QBwtYK7OIJ_1TosW9Dg4nmJqhq8MYEwb/view?usp=drive_link



Laboratory 2:

FINITE STATE MACHINES

Binary Code

```
module lab2_tn2_binary (
    input [0:0]KEY,
    input [1:0]SW,
    output [9:0]LEDR
) ;
// 1. State declaration (one-hot encoding)
parameter
A = 4'b0000,
B = 4'b0001,
C = 4'b0010,
D = 4'b0011,
E = 4'b0100,
F = 4'b0101,
G = 4'b0110,
H = 4'b0111,
I = 4'b1000;

logic [3:0] present_state , next_state ;
// 2. State register
always_ff @ ( posedge KEY[0] , posedge SW[0] ) begin /* FF description*/
if (SW[0])
present_state <= A;
else
present_state <= next_state;
end
// 3. Next-state combinational logic
always_comb /*next state transition*/
begin
next_state = present_state; // default state is the same
case ( present_state )
A : if ( SW[1] ) next_state = F;
else next_state = B;
B : if ( SW[1] ) next_state = F;
else next_state = C;
C : if ( SW[1] ) next_state = F;
else next_state = D;
D : if ( SW[1] ) next_state = F;
else next_state = E;
E : if ( SW[1] ) next_state = F;
else next_state = E;
F : if ( SW[1] ) next_state = G;
else next_state = B;
G : if ( SW[1] ) next_state = H;
```



Laboratory 2:

FINITE STATE MACHINES

```
else next_state = B;
H : if ( SW[1] ) next_state = I;
else next_state = B;
I : if ( SW[1] ) next_state = I;
else next_state = B;
default: next_state = A;
endcase
end
// 4. Output logic
assign LEDR[9] = (present_state == E) || (present_state == I);
assign LEDR[8:4] = 5'b00000;
assign LEDR[3:0] = present_state;
endmodule
```

Testbench

```
`timescale 1ns / 1ps

module lab2_tn2_binary_tb();

    // Khai báo các tín hiệu kết nối với module cần test (DUT)
    logic clk;
    logic rst; // Reset tích cực mức thấp (dựa theo hình ảnh)
    logic w;
    logic z;
    logic [3:0] state; // Dùng để quan sát trạng thái (Y)
    logic [9:0] ledr; // Khai báo mảng 10 bit, để có thể cho z = LEDR9,
state = LEDR[3:0]

    assign z = ledr[9];
    assign state = ledr[3:0];
    lab2_tn2_binary DUT (
        .KEY(clk), // Nối tín hiệu clk của testbench vào KEY[0]
        .SW({w, rst}), // Nối w vào SW[1] và rst vào SW[0]
        .LEDR(ledr) // Nối ngõ ra z vào LEDR[9] và trạng thái vào LEDR[3:0]
    );

    // Tạo xung Clock (Chu kỳ 10ns -> Tần số 100MHz)
    initial begin
        clk = 0;
        forever #5 clk = ~clk;
    end

    // Khởi tạo tín hiệu kích thích (Stimulus)
    initial begin
```



Laboratory 2:

FINITE STATE MACHINES

```
// 1. Khởi tạo ban đầu
rst = 1'b1;           // Kéo rst lên 1 để reset về trạng thái A
w = 1'b0;

// 2. Kích hoạt Reset (Active-low) ở đầu mô phỏng
@(negedge clk); // Đợi cạnh xuống của clock
rst = 1'b0;

@(negedge clk);
rst = 1'b0;           // FSM bắt đầu hoạt động

// 3. w = 0 trong khoảng 2 chu kỳ tiếp theo (tổng là 3 chu kỳ đầu)
@(negedge clk);
@(negedge clk);

// 4. w = 1 trong 1 chu kỳ
w = 1'b1;
@(negedge clk);

// 5. w = 0 trong 3 chu kỳ
w = 1'b0;
@(negedge clk);
@(negedge clk);
@(negedge clk);
@(negedge clk);

// 6. w = 1 trong 5 chu kỳ liên tiếp
// Đủ 4 chu kỳ thì z sẽ nhảy lên 1, chu kỳ thứ 5 z vẫn giữ mức 1
w = 1'b1;
@(negedge clk); // 1
@(negedge clk); // 2
@(negedge clk); // 3
@(negedge clk); // 4 -> Sau cạnh lên tiếp theo z sẽ = 1
@(negedge clk); // 5 -> z tiếp tục giữ 1

// 7. w = 0 trở lại, ngắt chuỗi 1 -> z sẽ rớt xuống 0
w = 1'b0;
@(negedge clk);
rst = 1'b1;           // Kéo rst lên 1
@(negedge clk);
rst = 1'b0;           // Máy chạy bth
@(negedge clk);
@(negedge clk);

// 8. Test thử reset giữa chừng
```

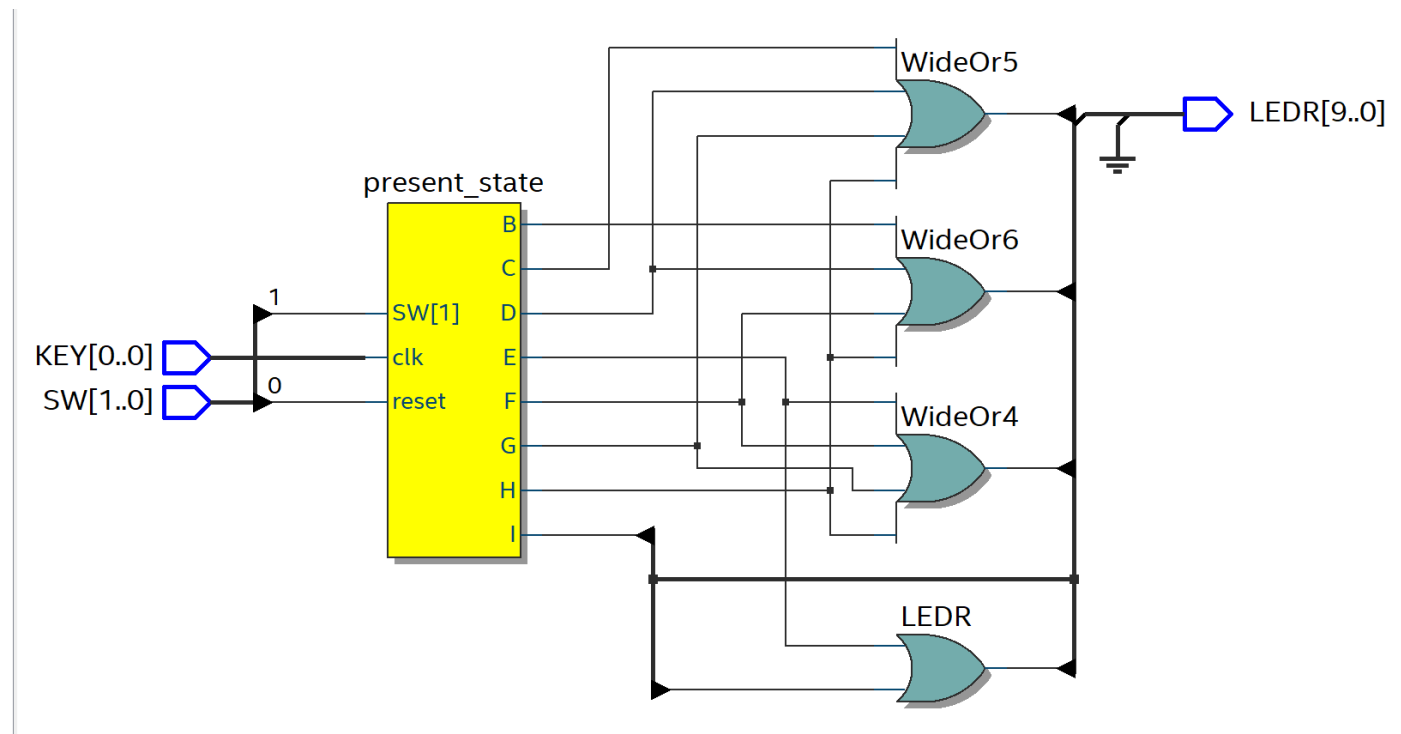


Laboratory 2:

FINITE STATE MACHINES

```
// Đợi thêm vài chu kỳ rồi kết thúc mô phỏng  
@ (negedge clk);  
@ (negedge clk);  
$stop;           // Dừng mô phỏng  
  
end  
  
endmodule
```

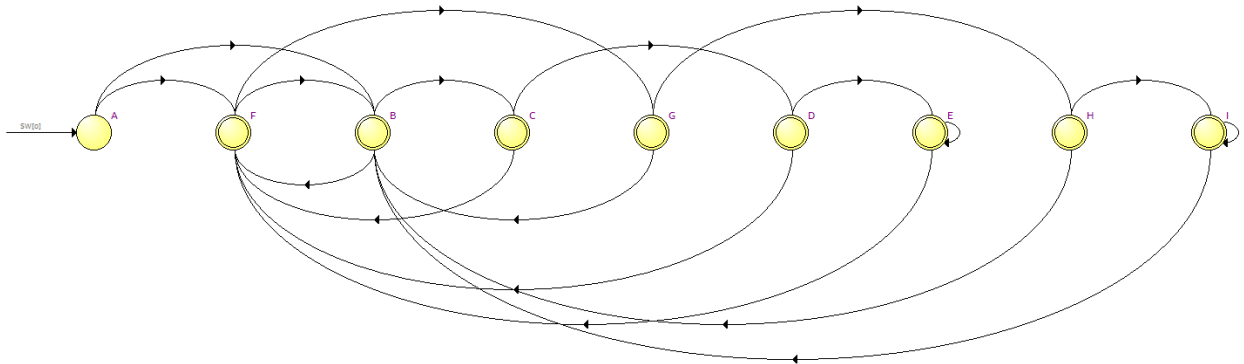
RTL



Laboratory 2:

FINITE STATE MACHINES

State machine

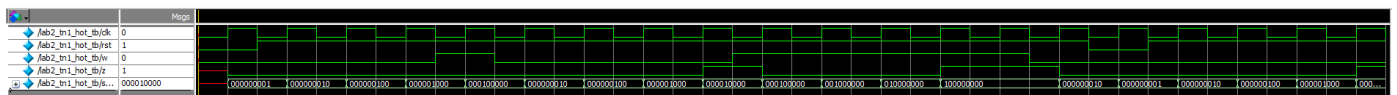


State Machine - [lab2_tn2_binary]present_state

<<Filter>>

	Name	present_state.H	present_state.G	present_state.F	present_state.E	present_state.D	present_state.C	present_state.B	present_state.A	present_s
1	present_state.A	0	0	0	0	0	0	0	0	0
2	present_state.B	0	0	0	0	0	1	1	1	0
3	present_state.C	0	0	0	0	1	0	1	1	0
4	present_state.D	0	0	0	1	0	0	1	1	0
5	present_state.E	0	0	1	0	0	0	1	1	0
6	present_state.F	0	1	0	0	0	0	1	1	0
7	present_state.G	0	0	0	0	0	0	1	1	0
8	present_state.H	1	0	0	0	0	0	1	1	0
9	present_state.I	0	0	0	0	0	0	1	1	1

Simulation



Experimental results VIDEO:

https://drive.google.com/file/d/1gZ9S5S2bl2PwTuWmSXtZ7gHL0AbiXXdc/view?usp=drive_link



Laboratory 2:

FINITE STATE MACHINES

EXPERIMENT 3

Objective: Analyze and implement the sequence detectors circuit by integrating shift registers and combinational logic circuits.

Requirement: Design and implement a sequence detector in SystemVerilog by instantiating two 4-bit shift registers. One shift register shall detect four consecutive 0s, and the other shall detect four consecutive 1s. Use appropriate combinational logic to produce the output signal z when either sequence is detected.

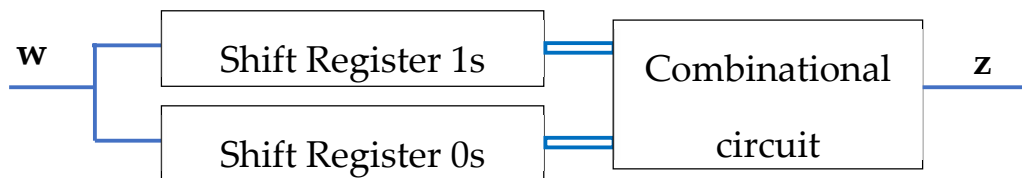


Figure 4: Sequence detector using shift registers.

Instruction:

1. Select an appropriate combinational circuit and illustrate the corresponding circuit design. The design must include synchronous reset signals, denoted as *rst*.
2. Write System Verilog code to describe your design (You may use the code in Exercise 3 of Pre Lab 2 to describe shift registers).
3. Use the same switches, push buttons, and lights that were used in Experiment 1.



Laboratory 2:

FINITE STATE MACHINES

4. Compile your project. Download the circuit into the FPGA chip and test its functionality. Observe the behavior of your shift registers and the output z.

Check: Your report must show the results:

- The figure illustrates your design.
- The waveform to prove the circuit works correctly.
- The result of RTL viewer.
- Experimental results obtained from running the design on the kit (the submitted photos must clearly show the student ID cards of all group members).

Answer the question: Could you use just one 4-bit shift register, rather than two? Explain your answer.

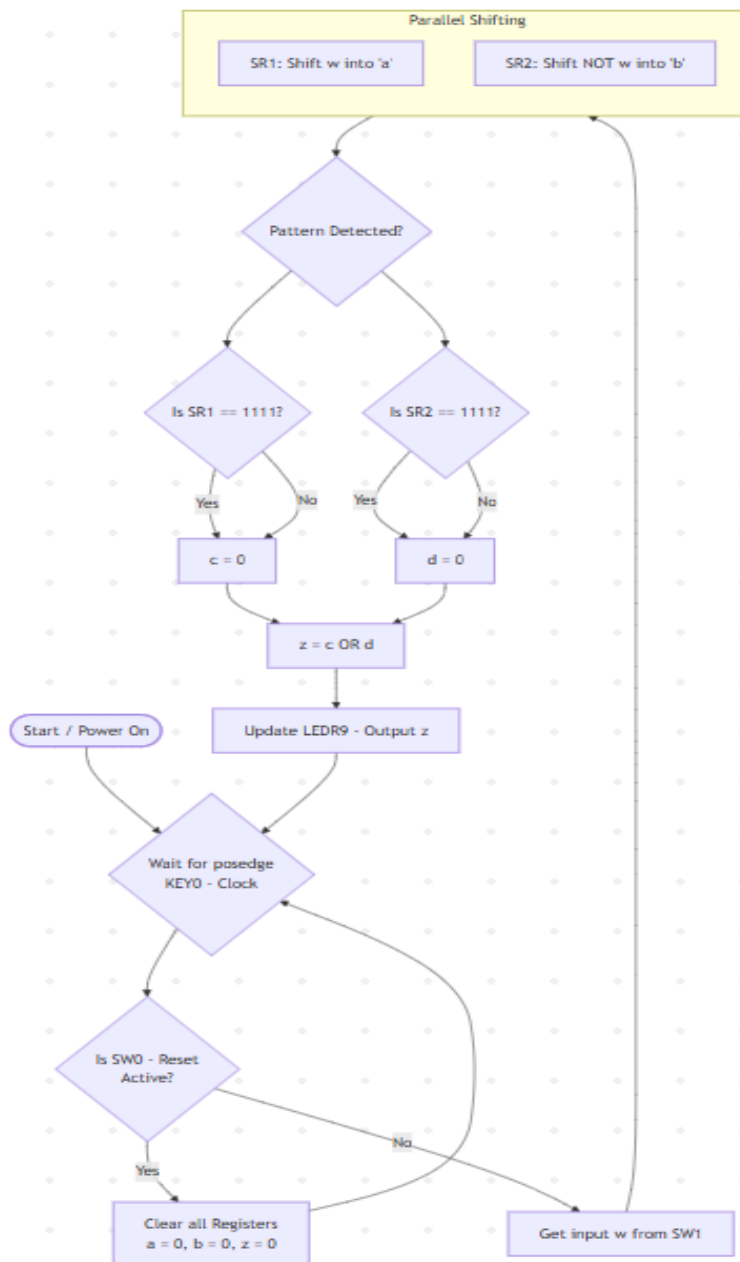
We can use only one 4-bit shift register because both original registers receive the same input w and store the exact same bit history. Using two registers is redundant, so we can save hardware resources by sharing a single register to track the input bits. We then apply combinational logic to the outputs of this single register to detect the required patterns. Specifically, we use an AND gate for the "1111" sequence and a NOR gate for the "0000" sequence. Finally, we combine these results with an OR gate to produce the output z correctly.



Laboratory 2:

FINITE STATE MACHINES

Design flowchart:



Laboratory 2:

FINITE STATE MACHINES

Code:

```
module d_ff(  
    input rst, clk, D,  
    output logic Q  
);  
    always_ff @(posedge clk)  
        begin  
            if (~rst)  
                Q <= 0;  
            else  
                Q <= D;  
            end  
        endmodule  
  
module shift_register  
(  
    input i, clk, rst,  
    output L1, L2, L3, L4  
);  
    d_ff D1 (  
        .clk(clk),  
        .rst(rst),  
        .D(i),  
        .Q(L1)  
    );  
    d_ff D2 (  
        .clk(clk),  
        .rst(rst),  
        .D(L1),  
        .Q(L2)  
    );  
    d_ff D3 (  
        .clk(clk),  
        .rst(rst),  
        .D(L2),  
        .Q(L3)  
    );  
    d_ff D4 (  
        .clk(clk),  
        .rst(rst),  
        .D(L3),  
        .Q(L4)  
    );  
endmodule
```



Laboratory 2:

FINITE STATE MACHINES

```
module lab2_tn3(
    input [1:0] SW,
    input [0:0] KEY,
    output [9:0] LEDR
);
    logic [3:0] a;
    logic [3:0] b;
    logic c, d;
    shift_register SR1 (
        .clk(KEY[0]),
        .rst(SW[0]),
        .i(SW[1]),
        .L1(a[0]),
        .L2(a[1]),
        .L3(a[2]),
        .L4(a[3])
    );
    shift_register SR2 (
        .clk(KEY[0]),
        .rst(SW[0]),
        .i(~SW[1]),
        .L1(b[0]),
        .L2(b[1]),
        .L3(b[2]),
        .L4(b[3])
    );
    assign c= a[0] & a[1] & a[2] & a[3];
    assign d= b[0] & b[1] & b[2] & b[3];
    assign LEDR [9] = c | d;
    assign LEDR [8] = 1'b0;
    assign LEDR [3:0] =a;
    assign LEDR [7:4] =b;
endmodule
```



Laboratory 2:

FINITE STATE MACHINES

Testbench

```
`timescale 1ns / 1ps

module lab2_tn3_tb();

    // Khai báo các tín hiệu kết nối với module cần test (DUT)
    logic clk;
    logic rst; // Reset tích cực mức thấp (dựa theo hình ảnh)
    logic w;
    logic z;
    logic [7:0] state; // Dùng để quan sát trạng thái (Y)

    // 2. Khai báo biến 10-bit để hứng toàn bộ ngõ ra từ mạch
    logic [9:0] LEDR_tb;
    assign z = LEDR_tb[9]; // Lấy bit 9 gán cho z
    assign state = LEDR_tb[7:0]; // Lấy 8 bit thấp gán cho state

    // Gọi module (DUT - Device Under Test)
    // Lưu ý: Thay "prelab2_tn1" bằng tên module thực tế của bạn nếu khác
    lab2_tn3 DUT (
        .KEY(clk), // Nối tín hiệu clk của testbench vào KEY[0]
        .SW({w, rst}), // Nối w vào SW[1] và rst vào SW[0]
        .LEDR(LEDR_tb) // Nối ngõ ra z vào LEDR[9] và trạng thái vào
        LEDR[7:0]
    );

    // Tạo xung Clock (Chu kỳ 10ns -> Tần số 100MHz)
    initial begin
        clk = 0;
        forever #5 clk = ~clk;
    end

    // Khởi tạo tín hiệu kích thích (Stimulus)
    initial begin
        // 1. Khởi tạo ban đầu
        rst = 1'b0;
        w = 1'b0;

        // 2. Kích hoạt Reset (Active-low) ở đầu mô phỏng
        @(negedge clk); // Đợi cạnh xuống của clock
        rst = 1'b1; // Kéo rst xuống 0 để reset về trạng thái A

        @(negedge clk);
        rst = 1'b1; // Nhả reset (lên 1), FSM bắt đầu hoạt động
    end
endmodule
```



Laboratory 2:

FINITE STATE MACHINES

```
// 3. w = 0 trong khoảng 2 chu kỳ tiếp theo (tổng là 3 chu kỳ đầu)
@(negedge clk);
@(negedge clk);

// 4. w = 1 trong 1 chu kỳ
w = 1'b1;
@(negedge clk);

// 5. w = 0 trong 3 chu kỳ
w = 1'b0;
@(negedge clk);
@(negedge clk);
@(negedge clk);
@(negedge clk);

// 6. w = 1 trong 5 chu kỳ liên tiếp
// Dù 4 chu kỳ thì z sẽ nhảy lên 1, chu kỳ thứ 5 z vẫn giữ mức 1
w = 1'b1;
@(negedge clk); // 1
@(negedge clk); // 2
@(negedge clk); // 3
@(negedge clk); // 4 -> Sau cạnh lên tiếp theo z sẽ = 1
@(negedge clk); // 5 -> z tiếp tục giữ 1

// 7. w = 0 trở lại, ngắt chuỗi 1 -> z sẽ rớt xuống 0
w = 1'b0;
@(negedge clk);
    rst = 1'b0;        // Kéo rst xuống 0
@(negedge clk);
rst = 1'b1;           // Nhả reset
@(negedge clk);
@(negedge clk);

// 8. Test thử reset giữa chừng

// Đợi thêm vài chu kỳ rồi kết thúc mô phỏng
@(negedge clk);
@(negedge clk);
$stop;                // Dừng mô phỏng

End

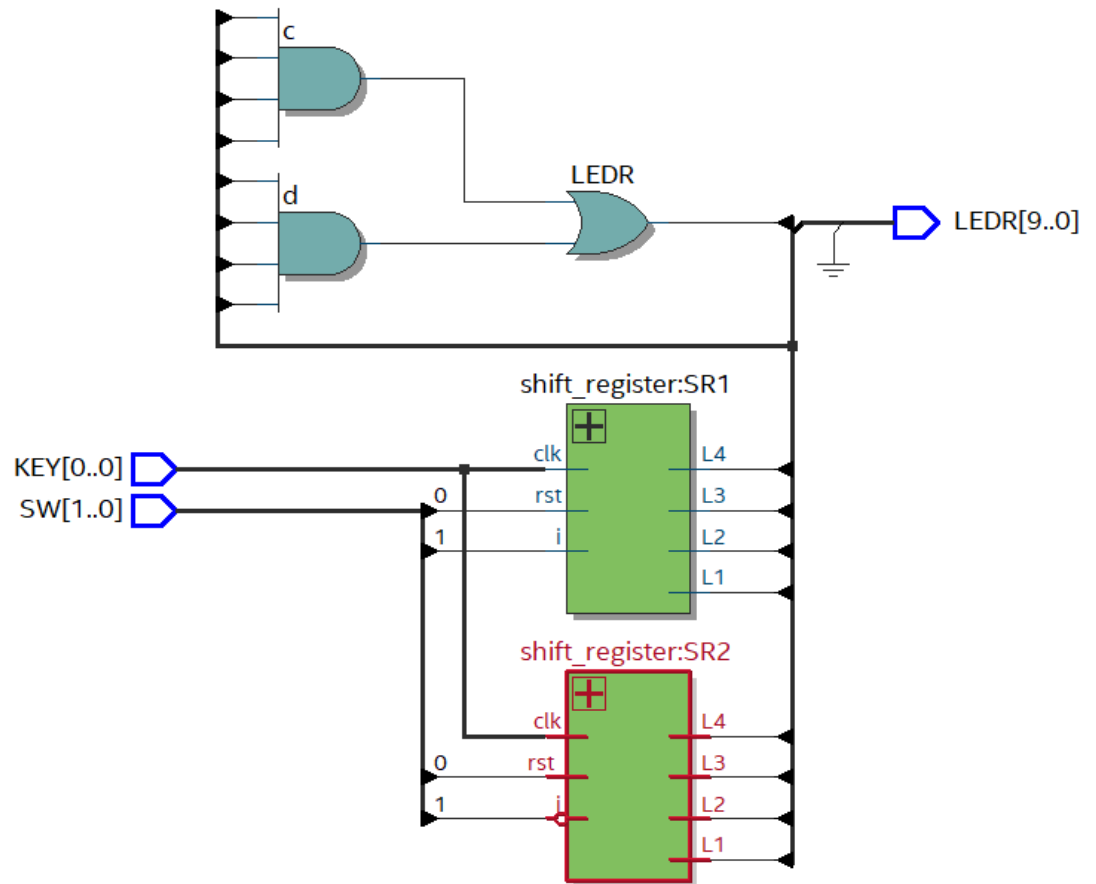
endmodule
```



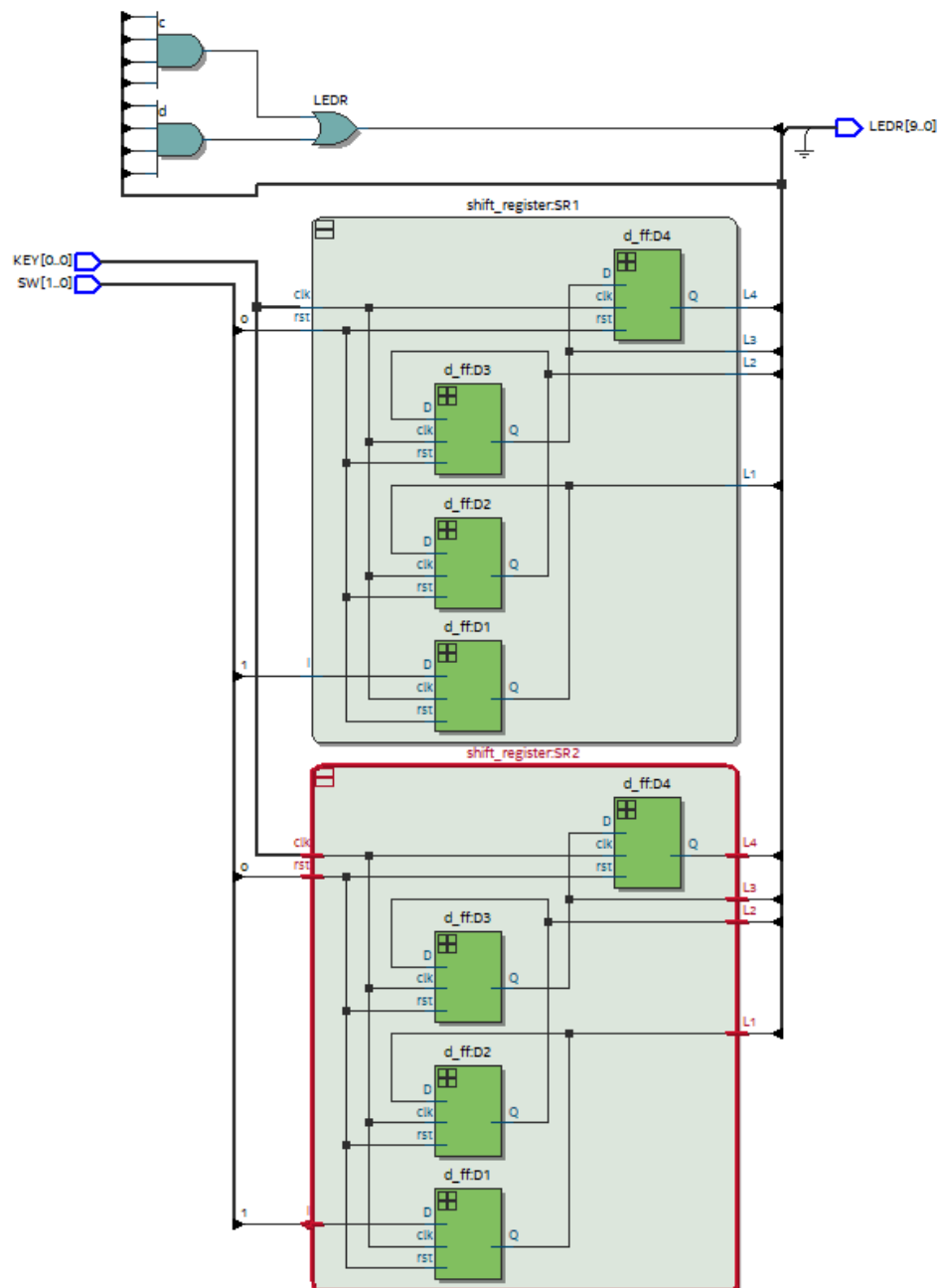
Laboratory 2:

FINITE STATE MACHINES

RTL



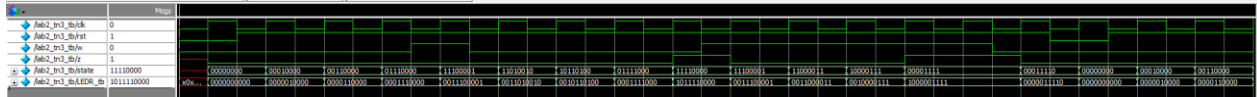
Laboratory 2: FINITE STATE MACHINES



Laboratory 2:

FINITE STATE MACHINES

Simulation



Experimental results VIDEO:

[https://drive.google.com/file/d/1GBoRUN8S5iCRi5UIL1USIFQFjt0p8COA/view?usp=drive link](https://drive.google.com/file/d/1GBoRUN8S5iCRi5UIL1USIFQFjt0p8COA/view?usp=drive_link)



Laboratory 2:

FINITE STATE MACHINES

EXPERIMENT 4

Objective: Design and implement a digital circuit using FSM.

Requirement: The Morse code uses patterns of short and long pulses to represent a message. Each letter is represented as a sequence of dots (a short pulse), and dashes (a long pulse). For example, the first eight letters of the alphabet have the following representation:

A • —
B — • • •
C — • — •
D — • •
E •
F • • — •
G — — •
H • • • •

Design and implement a Morse-code encoder circuit using an FSM. The circuit takes as input one of the first eight letters of the alphabet and displays the Morse code for it on LEDs.

A high-level schematic diagram of a possible circuit for the Morse-code encoder is shown in Figure 5.



Laboratory 2:

FINITE STATE MACHINES

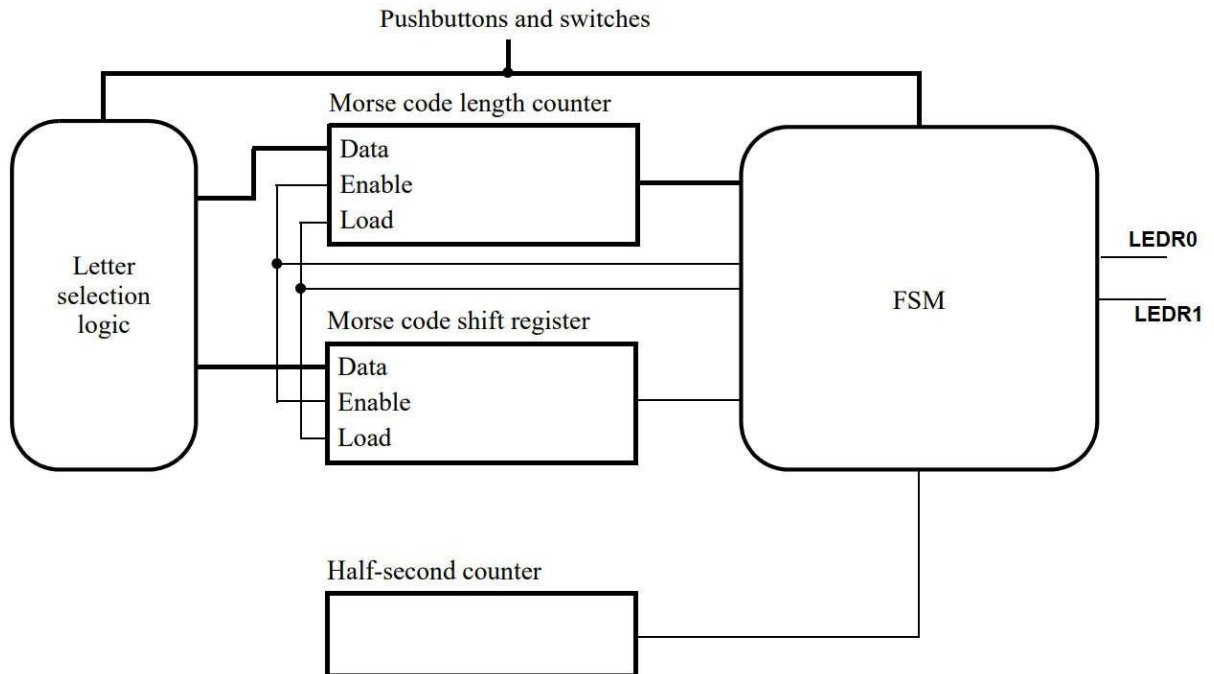


Figure 5: High-level schematic diagram for the Morse-code encoder.

Instruction:

1. Filling this suggested skeleton of the System Verilog code:



Laboratory 2:

FINITE STATE MACHINES

```
module part4 (SW, CLOCK_50, KEY, LEDR);

    /******
    /***  DECLARATIONS  **
    /******

    // FSM State Table
    always_comb @(...)
    begin: state_table
        ...
    end // state_table

    // FSM State flip-flops

    always_ff @(posedge Clock)
        ...
```



Laboratory 2:

FINITE STATE MACHINES

```
// FSM outputs
// turn on the Morse code light in the states below
assign light_on = ... ;

// specify when to load the Morse code into the shift register, and length into the counter
assign load_regs = ...;

// specify when to shift the Morse code bits and decrease the length counter
assign shift_and_count = ...;

/* Create an enable signal that is asserted once every second */
modulo_counter ...;

/* Letter selection
*/
always_comb @(*)
case (SW)
    A_SW: begin
        morse_code = ... ;
morse_length = ... ;          end
    ...
    ...

endcase

/* Store the Morse code to be sent in a shift register, and its length in a counter */
always_ff @ (posedge CLOCK_50)
begin

    /* if Reset = 0 then data = size = 0;
    otherwise, if load = 1 then data = morse_code and size = morse_length;
    if shift = 1 then data[2:0]= data[3:1]) & data[3] = 1'b0 and size = size - 1'b1 */

endmodule
```



Laboratory 2:

FINITE STATE MACHINES

```
module modulo_counter(...);  
    ...  
  
endmodule
```



Laboratory 2:

FINITE STATE MACHINES

2. Select an appropriate method and illustrate the corresponding circuit design.
The design must include asynchronous reset signals, denoted as *rst*.
3. Write System Verilog code to describe your design.
4. Use switches SW2 – SW0 and pushbu ons KEY1 – KEY0 as inputs. When a user presses KEY1, the circuit should display the Morse code for a letter specified by SW₂₋₀, using a LEDR0 to represent dots, and LEDR1 to represent dashes. Each LED will be on for half of a second. Pushbutton KEY0 should function as an asynchronous reset.

Check: Your report must show the results:

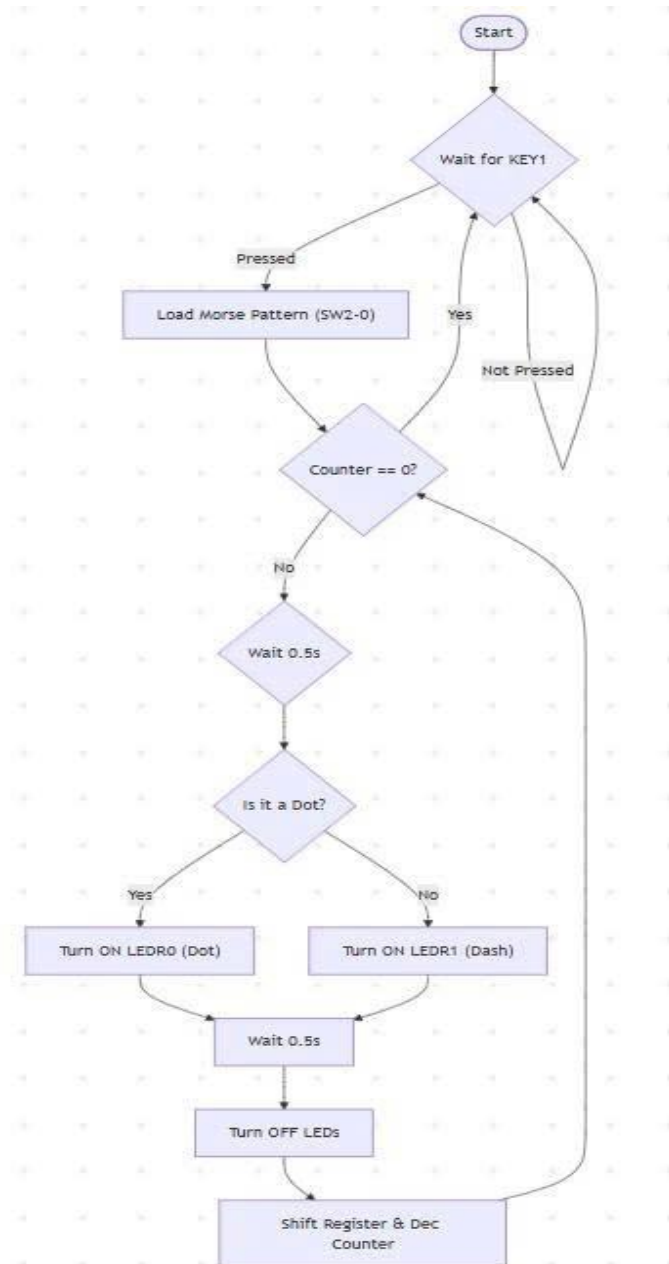
- The figure illustrates your design.
- The waveform to prove the circuit works correctly.
- The result of RTL viewer.
- Experimental results obtained from running the design on the kit (the submitted photos must clearly show the student ID cards of all group members).



Laboratory 2:

FINITE STATE MACHINES

The figure illustrates your design:



Laboratory 2:

FINITE STATE MACHINES

Code

```
//
=====
// MODULE TOP: Gắn kết các khối lại với nhau giống như sơ đồ mạch
//
=====
module lab2_tn4_ver2(
    input logic [2:0] SW,           // Pushbuttons and switches
    input logic      CLOCK_50,     // Xung clock 50MHz
    input logic [1:0] KEY,         // Pushbuttons and switches
    output logic [9:0] LEDR        // LED outputs
);
    // Tắt các LED không sử dụng
    assign LEDR[9:2] = 8'h0;

    logic reset_n, start;
    assign reset_n = KEY[0];
    assign start   = ~KEY[1];

    // --- CÁC ĐƯỜNG DÂY TÍN HIỆU KẾT NỐI GIỮA CÁC KHỐI (Dây nối trong hình) ---
    logic [3:0] w_morse_data; // Từ Letter Selection -> Shift Register
    logic [2:0] w_morse_length; // Từ Letter Selection -> Length Counter

    logic w_fsm_load;           // Từ FSM -> Load của các thanh ghi
    logic w_fsm_enable;         // Từ FSM -> Enable của các thanh ghi
    logic w_half_sec_tick;      // Từ Half-second counter -> FSM

    logic [2:0] w_current_size; // Từ Length Counter -> FSM
    logic w_current_bit;        // Từ Shift Register -> FSM (Bit LSB)

    // -----
    // KHỐI 1: Letter selection logic
    // -----
    letter_selection_logic letter_logic_inst (
        .SW(SW),
        .morse_code(w_morse_data),
        .morse_length(w_morse_length)
    );

    // -----
    // -----
```



Laboratory 2:

FINITE STATE MACHINES

```
// KHỐI 2: Morse code length counter
// -----

morse_code_length_counter length_counter_inst (
    .clk(CLOCK_50),
    .rst_n(reset_n),
    .data(w_morse_length),
    .load(w_fsm_load),
    .enable(w_fsm_enable),
    .count_out(w_current_size)
);

// -----

// KHỐI 3: Morse code shift register
// -----

morse_code_shift_register shift_register_inst (
    .clk(CLOCK_50),
    .rst_n(reset_n),
    .data(w_morse_data),
    .load(w_fsm_load),
    .enable(w_fsm_enable),
    .bit_out(w_current_bit)
);

// -----

// KHỐI 4: Half-second counter
// -----

half_second_counter #(.MAX_COUNT(25_000_000)) half_sec_cnt(
    .clk(CLOCK_50),
    .rst_n(reset_n),
    .tick(w_half_sec_tick)
);

// -----

// KHỐI 5: FSM (Finite State Machine)
// -----

fsm_controller fsm_inst (
    .clk(CLOCK_50),
    .rst_n(reset_n),
    .start(start),
```



Laboratory 2:

FINITE STATE MACHINES

```
.tick(w_half_sec_tick),
.current_size(w_current_size),
.current_bit(w_current_bit),
.fsm_load(w_fsm_load),
.fsm_enable(w_fsm_enable),
.ledr0(LED0[0]),
.ledr1(LED0[1])
);

endmodule

//
=====
// CHI TIẾT CÁC SUB-MODULES
//
=====

// --- KHỐI 1: Letter selection logic ---

module letter_selection_logic (
    input logic [2:0] SW,
    output logic [3:0] morse_code,
    output logic [2:0] morse_length
);
    always_comb begin
        case (SW)
            3'b000: begin morse_code = 4'b0010; morse_length = 3'd2; end // A:
            3'b001: begin morse_code = 4'b0001; morse_length = 3'd4; end // B:
            3'b010: begin morse_code = 4'b0101; morse_length = 3'd4; end // C:
            3'b011: begin morse_code = 4'b0001; morse_length = 3'd3; end // D:
            3'b100: begin morse_code = 4'b0000; morse_length = 3'd1; end // E:
            3'b101: begin morse_code = 4'b0100; morse_length = 3'd4; end // F:
            3'b110: begin morse_code = 4'b0011; morse_length = 3'd3; end // G:
            3'b111: begin morse_code = 4'b0000; morse_length = 3'd4; end // H:
            default: begin morse_code = 4'b0000; morse_length = 3'd0; end
        endcase
    end
end
```



Laboratory 2:

FINITE STATE MACHINES

```
endmodule

// --- KHỐI 2: Morse code length counter ---
module morse_code_length_counter (
    input logic clk,
    input logic rst_n,
    input logic [2:0] data,
    input logic load,
    input logic enable,
    output logic [2:0] count_out
);
    always_ff @(posedge clk or negedge rst_n) begin
        if (!rst_n)
            count_out <= 3'd0;
        else if (load)
            count_out <= data;
        else if (enable)
            count_out <= count_out - 3'd1;
    end
endmodule

// --- KHỐI 3: Morse code shift register ---
module morse_code_shift_register (
    input logic clk,
    input logic rst_n,
    input logic [3:0] data,
    input logic load,
    input logic enable,
    output logic bit_out
);
    logic [3:0] shift_reg;
    assign bit_out = shift_reg[0]; // Bit đang xét luôn nằm ở LSB

    always_ff @(posedge clk or negedge rst_n) begin
        if (!rst_n)
            shift_reg <= 4'd0;
        else if (load)
            shift_reg <= data;
        else if (enable)
            shift_reg <= {1'b0, shift_reg[3:1]}; // Dịch phải
    end
endmodule

// --- KHỐI 4: Half-second counter ---
module half_second_counter #(
    parameter MAX_COUNT = 25_000_000
```



Laboratory 2:

FINITE STATE MACHINES

```
) (
    input logic clk,
    input logic rst_n,
    output logic tick
);
    logic [24:0] count;
    always_ff @(posedge clk or negedge rst_n) begin
        if (!rst_n) begin
            count <= 25'd0;
            tick <= 1'b0;
        end else begin
            if (count == MAX_COUNT - 1) begin
                count <= 25'd0;
                tick <= 1'b1;
            end else begin
                count <= count + 1'b1;
                tick <= 1'b0;
            end
        end
    end
end
endmodule

// --- KHỐI 5: FSM (Tối ưu 3 Trạng thái) ---
module fsm_controller (
    input logic clk,
    input logic rst_n,
    input logic start,
    input logic tick,
    input logic [2:0] current_size,
    input logic current_bit,

    output logic fsm_load,
    output logic fsm_enable,
    output logic ledr0,
    output logic ledr1
);
    // Khai báo 3 trạng thái theo phương pháp Parameter
    parameter IDLE      = 2'b00,
               SENT_ON   = 2'b01,
               SENT_OFF  = 2'b10;

    logic [1:0] present_state, next_state;

    // 1. Logic chuyển trạng thái (Next State Logic)
    always_comb begin
        next_state = present_state; // Mặc định giữ nguyên trạng thái
```



Laboratory 2:

FINITE STATE MACHINES

```
case (present_state)
  IDLE: begin
    if (start) next_state = SENT_ON;
  end
  SENT_ON: begin
    if (tick) next_state = SENT_OFF;
  end
  SENT_OFF: begin
    if (tick) begin
      // Nếu còn 1 bit cuối cùng thì quay về IDLE, ngược lại
      tiếp tục sáng
      if (current_size <= 3'd1) next_state = IDLE;
      else next_state = SENT_ON;
    end
  end
  default: next_state = IDLE;
endcase
end

// 2. Thanh ghi trạng thái (State Register)
always_ff @(posedge clk or negedge rst_n) begin
  if (!rst_n) present_state <= IDLE;
  else present_state <= next_state;
end

// --- ĐIỂM MẪU CHỐT GỘP TRẠNG THÁI ---

// Kích hoạt LOAD ngay khi đang ở IDLE và có người bấm start
assign fsm_load = (present_state == IDLE) && start;

// Kích hoạt SHIFT (enable) khi đang ở chu kỳ tắt (SENT_OFF), hết nửa giây (tick) và vẫn còn bit để gửi
assign fsm_enable = (present_state == SENT_OFF) && tick && (current_size > 3'd1);

// Điều khiển LED tương ứng với bit truyền (0: dấu chấm, 1: dấu gạch)
assign ledr0 = (present_state == SENT_ON) & ~current_bit;
assign ledr1 = (present_state == SENT_ON) & current_bit;

endmodule
```



Laboratory 2:

FINITE STATE MACHINES

Testbench

```
`timescale 1ns / 1ps

module lab2_tn4_tb();

    // 1. Declare connection signals
    logic [2:0] SW;
    logic      CLOCK_50;
    logic [1:0] KEY;
    logic [9:0] LEDR;

    // Auxiliary signals for easier waveform viewing
    logic led_dot, led_dash;
    assign led_dot  = LEDR[0];
    assign led_dash = LEDR[1];

    // 2. Instantiate the design module
    lab2_tn4_ver2 DUT (
        .SW(SW),
        .CLOCK_50(CLOCK_50),
        .KEY(KEY),
        .LEDR(LEDR)
    );

    // Scale down the counter for simulation purposes
    defparam DUT.half_sec_cnt.MAX_COUNT = 50;

    // 3. Generate Clock (20ns period)
    initial begin
        CLOCK_50 = 0;
        forever #10 CLOCK_50 = ~CLOCK_50;
    end

    // 4. Simulation scenario: Sequential transmission from A to H
    initial begin
        // --- INITIALIZATION ---
        SW  = 3'b000;
        KEY = 2'b11;

        // Wait a few clock cycles for the system to stabilize
        repeat(5) @(posedge CLOCK_50);

        // --- SYSTEM STARTUP RESET ---
        $display("--- SYSTEM STARTUP ---");
        @(negedge CLOCK_50) KEY[0] = 1'b0; // Press Reset
    end
endmodule
```



Laboratory 2:

FINITE STATE MACHINES

```
repeat(2) @(posedge CLOCK_50); // Hold Reset for 2 cycles
@(negedge CLOCK_50) KEY[0] = 1'b1; // Release Reset
repeat(5) @(posedge CLOCK_50);

// =====
// CONTINUOUS TRANSMISSION FROM A TO H
// =====

// --- LETTER A (.-) ---
$display("Transmitting letter A (.-)");
@(negedge CLOCK_50) SW = 3'b000; KEY[1] = 1'b0;
repeat(2) @(posedge CLOCK_50);
@(negedge CLOCK_50) KEY[1] = 1'b1;
repeat(250) @(posedge CLOCK_50);

// --- LETTER B (-...) ---
$display("Transmitting letter B (-...)");
@(negedge CLOCK_50) SW = 3'b001; KEY[1] = 1'b0;
repeat(2) @(posedge CLOCK_50);
@(negedge CLOCK_50) KEY[1] = 1'b1;
repeat(450) @(posedge CLOCK_50);

// --- LETTER C (-.-) ---
$display("Transmitting letter C (-.-)");
@(negedge CLOCK_50) SW = 3'b010; KEY[1] = 1'b0;
repeat(2) @(posedge CLOCK_50);
@(negedge CLOCK_50) KEY[1] = 1'b1;
repeat(450) @(posedge CLOCK_50);

// --- LETTER D (-..) ---
$display("Transmitting letter D (-..)");
@(negedge CLOCK_50) SW = 3'b011; KEY[1] = 1'b0;
repeat(2) @(posedge CLOCK_50);
@(negedge CLOCK_50) KEY[1] = 1'b1;
repeat(350) @(posedge CLOCK_50);

// --- LETTER E (.) ---
$display("Transmitting letter E (.)");
@(negedge CLOCK_50) SW = 3'b100; KEY[1] = 1'b0;
repeat(2) @(posedge CLOCK_50);
@(negedge CLOCK_50) KEY[1] = 1'b1;
repeat(150) @(posedge CLOCK_50);

// --- LETTER F (..-) ---
$display("Transmitting letter F (..-)");
@(negedge CLOCK_50) SW = 3'b101; KEY[1] = 1'b0;
```



Laboratory 2:

FINITE STATE MACHINES

```
repeat(2) @(posedge CLOCK_50);
@(negedge CLOCK_50) KEY[1] = 1'b1;
repeat(450) @(posedge CLOCK_50);

// --- LETTER G (--) ---
$display("Transmitting letter G (--)");
@(negedge CLOCK_50) SW = 3'b110; KEY[1] = 1'b0;
repeat(2) @(posedge CLOCK_50);
@(negedge CLOCK_50) KEY[1] = 1'b1;
repeat(350) @(posedge CLOCK_50);

// --- LETTER H (....) ---
$display("Transmitting letter H (....)");
@(negedge CLOCK_50) SW = 3'b111; KEY[1] = 1'b0;
repeat(2) @(posedge CLOCK_50);
@(negedge CLOCK_50) KEY[1] = 1'b1;
repeat(450) @(posedge CLOCK_50);

// --- REST AND FINISH ---
repeat(100) @(posedge CLOCK_50);
$display("--- END OF SIMULATION ---");
$stop;
end

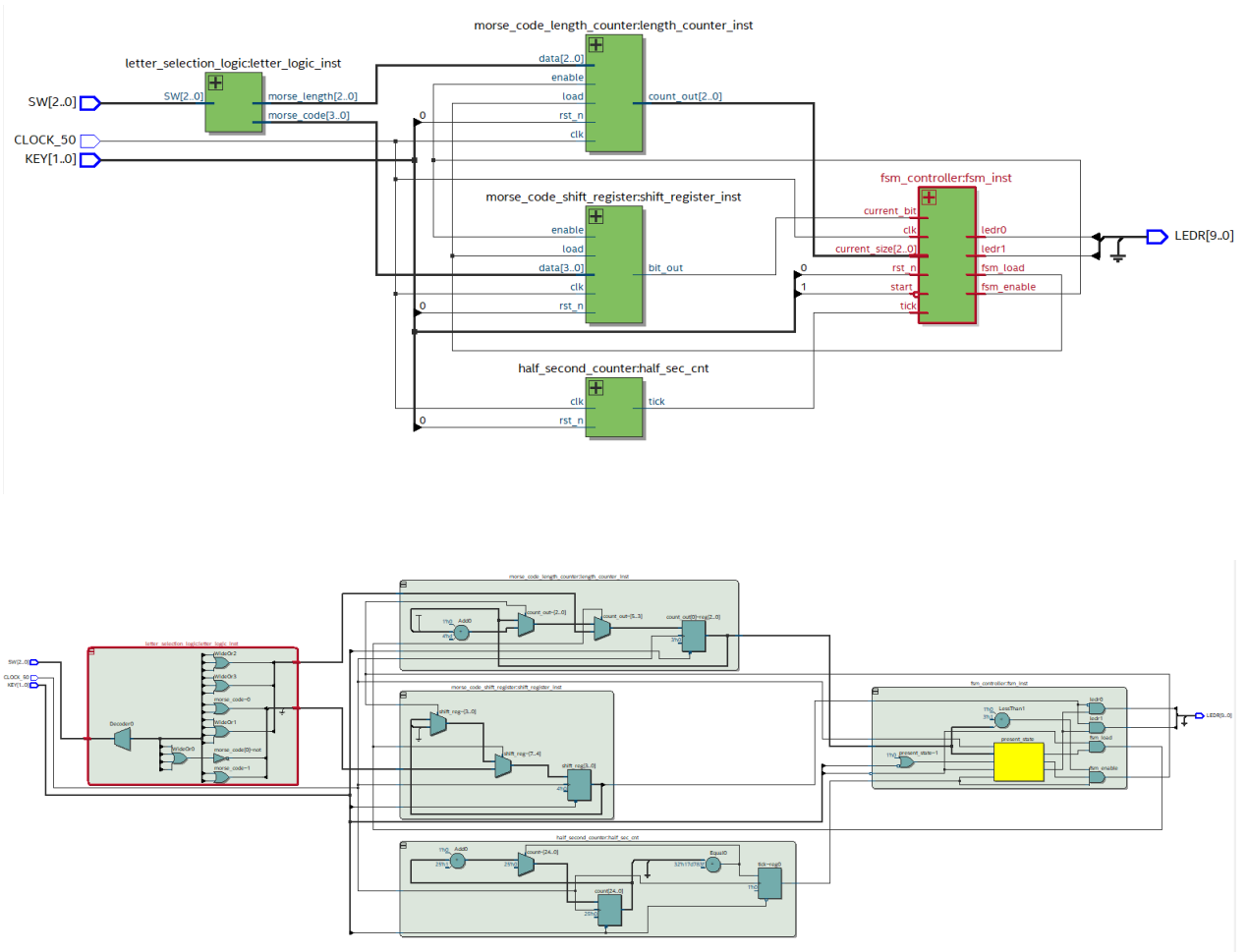
endmodule
```



Laboratory 2:

FINITE STATE MACHINES

RTL

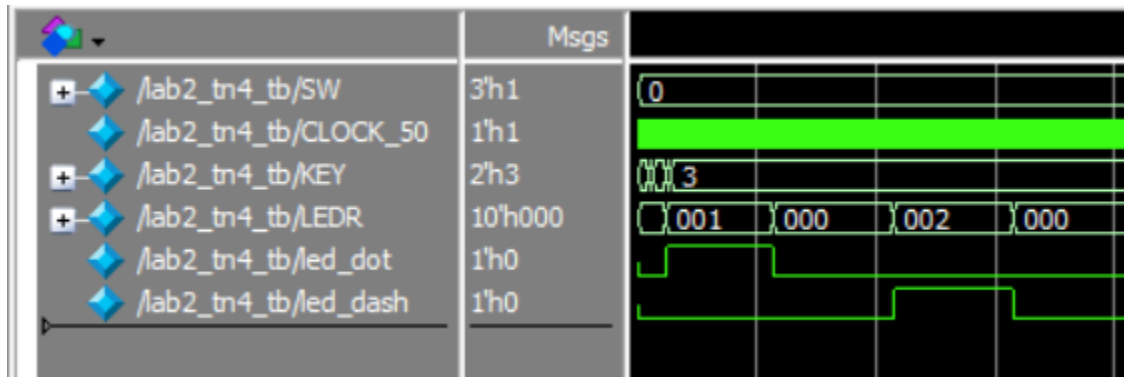


Laboratory 2:

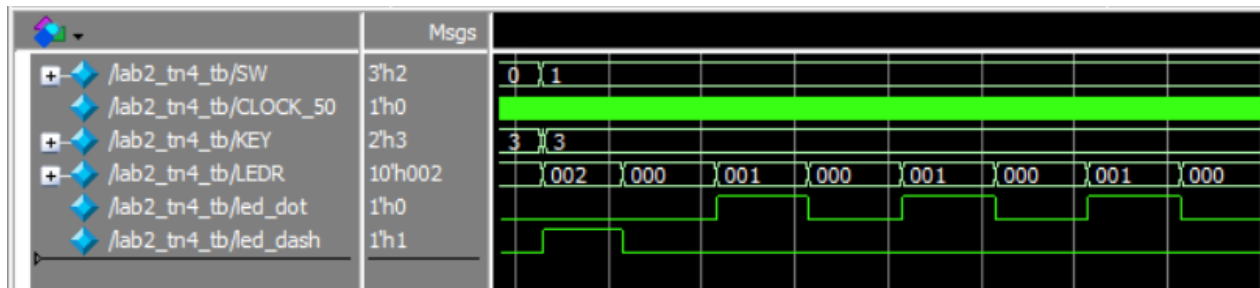
FINITE STATE MACHINES

Simulation

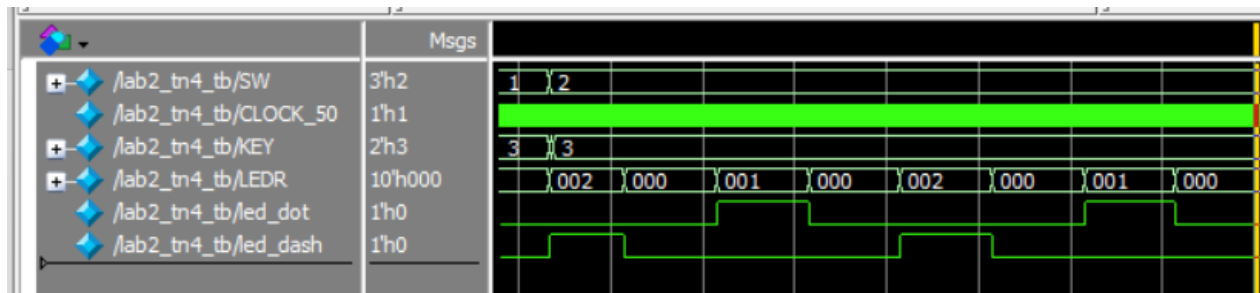
A:



B:



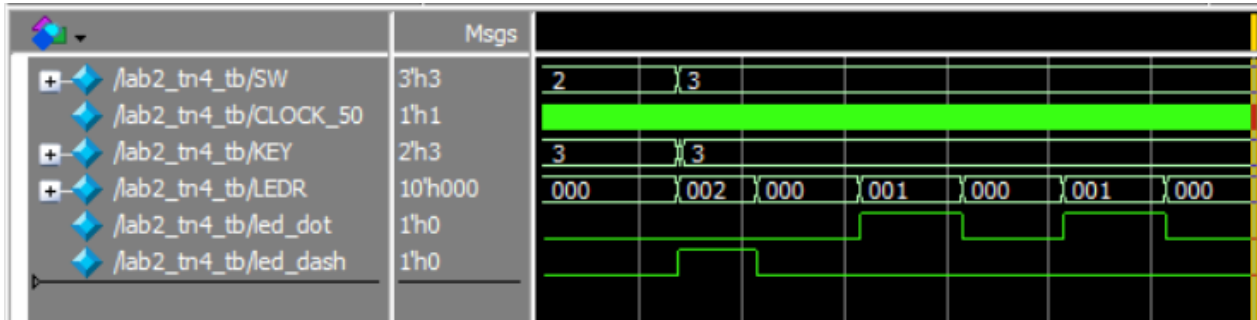
C:



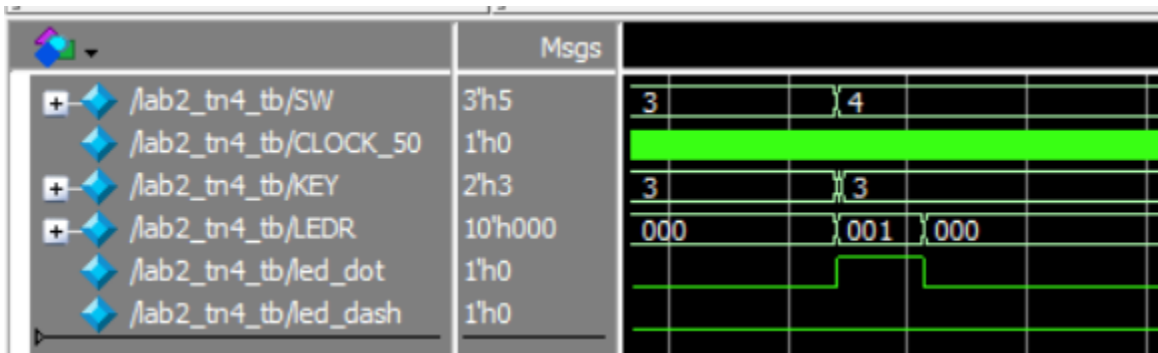
Laboratory 2:

FINITE STATE MACHINES

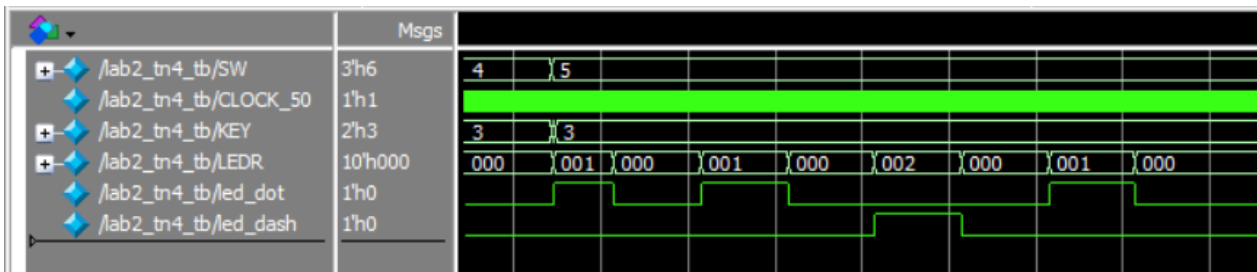
D:



E:



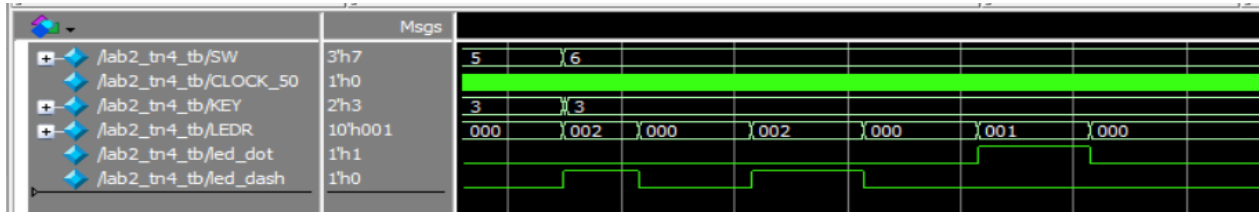
F:



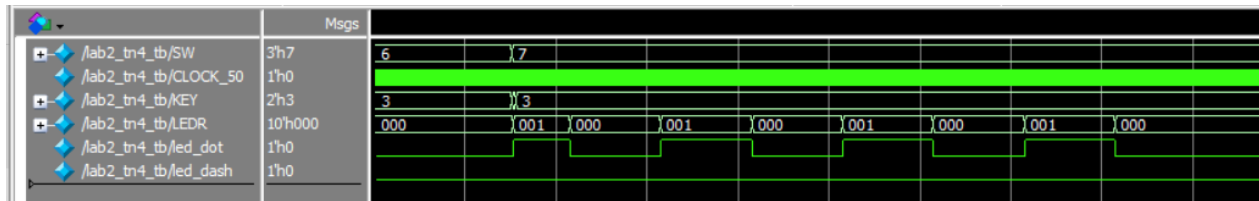
Laboratory 2:

FINITE STATE MACHINES

G:



H:



Experimental results VIDEO:

https://drive.google.com/file/d/1JzF51A2gpsjLUD4x7gt953g8qoGVP2pj/view?usp=drive_link

REFERENCES:

Link to folder contain all video:

https://drive.google.com/drive/folders/1n3AiP9ehY0xrvZDN1ukzKsBPLd4wn6Qh?usp=drive_link

Link to folder contain all code:

https://drive.google.com/drive/folders/1BMpzQTHbu5agt-QTqzQh8Ovzr1-h9dTq?usp=drive_link

